

딥러닝 기반 추천 시스템

모두의연구소

추천 시스템, 모델 튜닝하기: 논문과 코드

이수진

Contents

01

추천 시스템 논문을
학습하기 전에

1. 평가 방법
2. 활용 데이터 셋

02

NCF

1. 논문 리뷰
2. 코드 리뷰

03

DeepFM

1. 논문 리뷰
2. 코드 리뷰

04

AutoInt

1. 논문 리뷰
2. 코드 리뷰

05

HAFM

1. 논문 리뷰

01

추천 시스템 논문을 학습하기 전에

1. 평가 방법
2. 데이터 셋

1. 평가 방법

추천 시스템 논문을 보면,
다양한 평가 방법과 데이터 셋이
존재하는 것을 알 수 있습니다.

본격적인 논문 리뷰에 앞서, 평가 방법과 데이터 셋에 대해서 학습해보겠습니다.

1. 평가 방법

먼저, 평가 방법입니다.

평가 방법은 다음과 같이 구분됩니다.

1. 매칭 기반
2. 랭킹 기반
3. 오차 기반

1. 평가 방법

1. 매칭 기반 평가(Top K)

a. Precision@K

- “추천 시스템이 추천한 아이템 k개 중 실제 사용자가 관심있는 아이템 비율은?”의 의미이며 높을수록 좋음

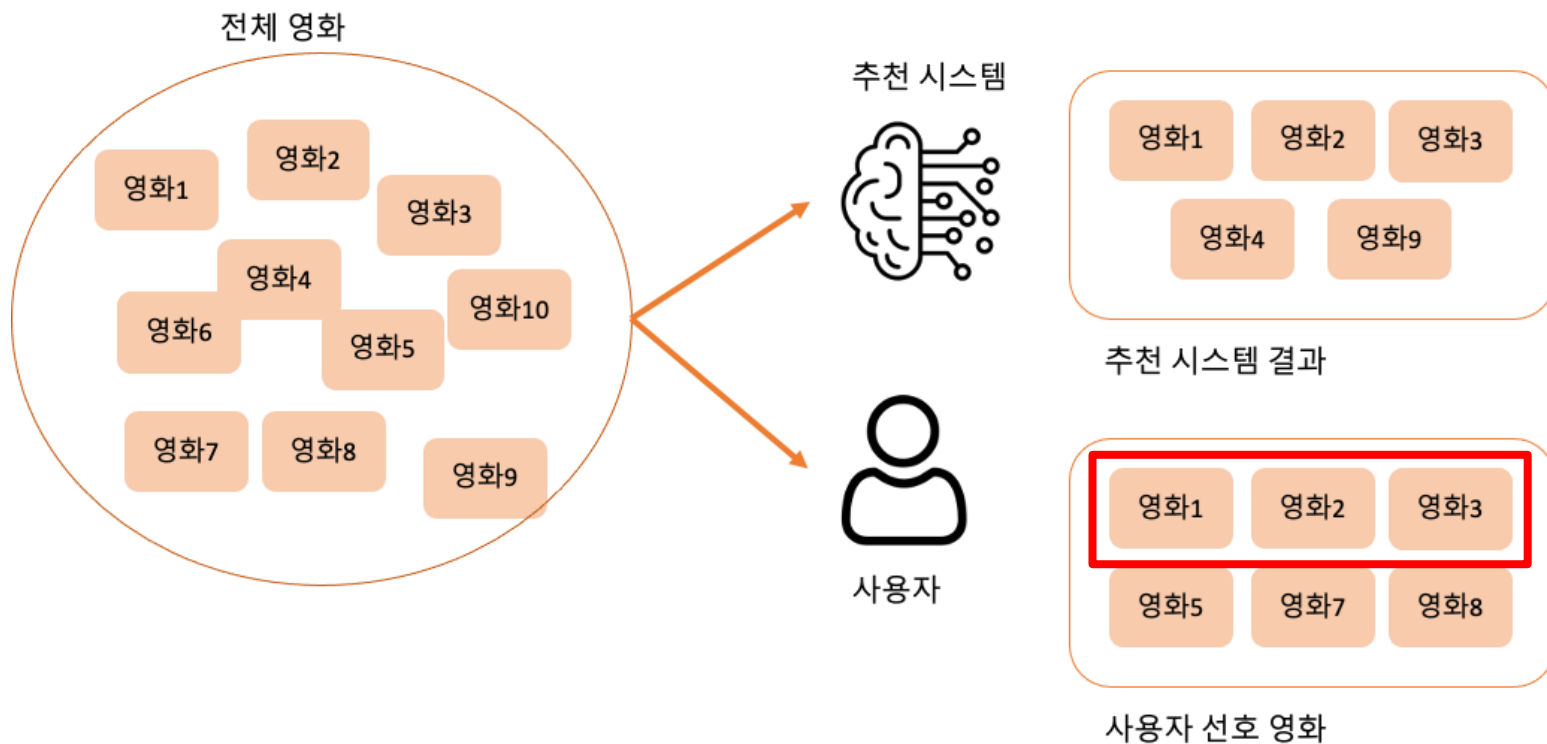
a. Recall@K

- “사용자가 관심있는 아이템 중 추천 시스템이 추천한 아이템 k개가 얼마나 포함되는가?”의 의미이며
높을수록 좋음

a. Hit Rate@K

- “추천 시스템이 추천한 아이템 k개 중 사용자가 관심있는 아이템이 1개라도 있으면 Hit”의 의미이며
높을수록 좋음

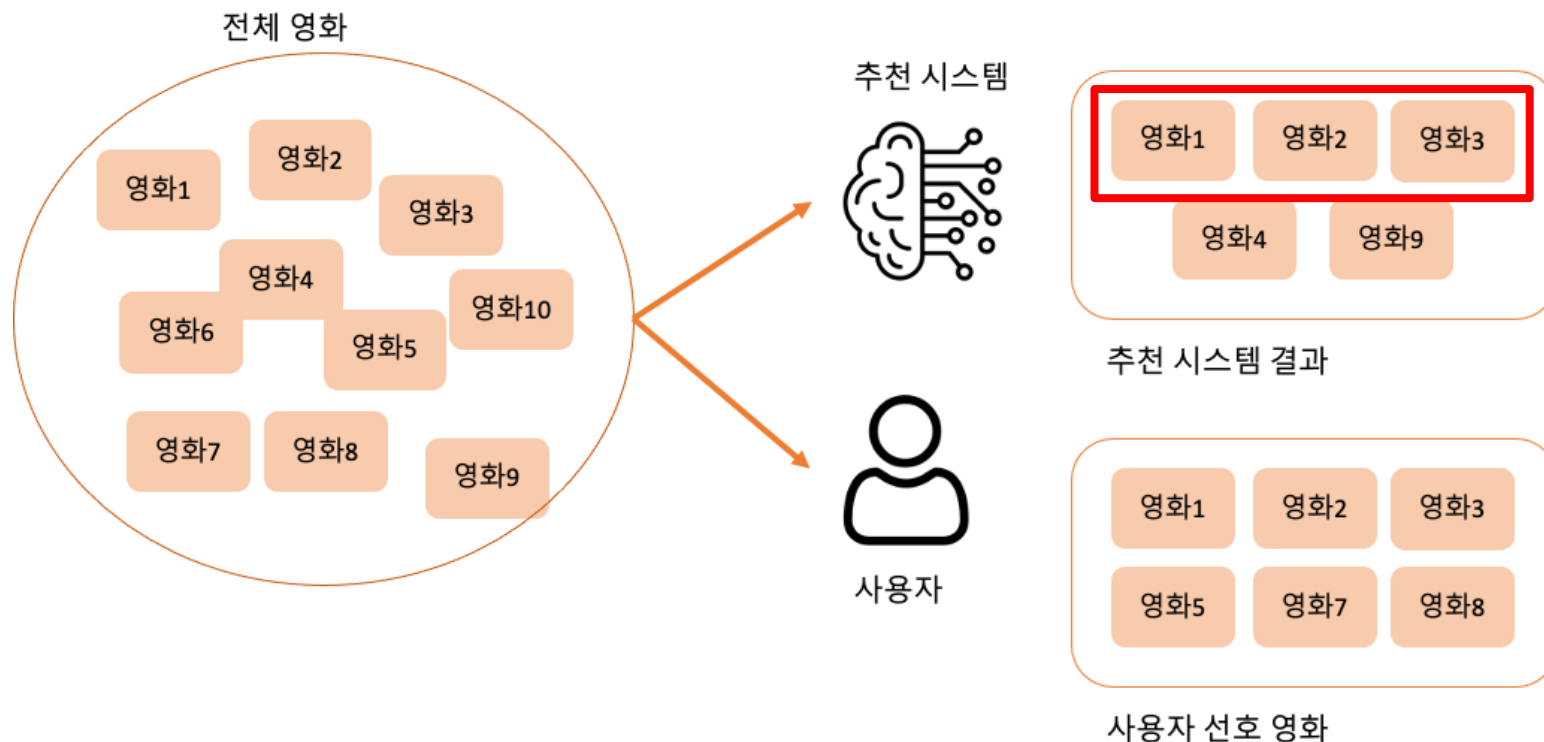
1. 평가 방법 - Precision@K



추천한 아이템 (1, 2, 3, 4, 9) 중 사용자가 선호하는 아이템 비율

$$Precision@5 = \frac{3}{5}$$

1. 평가 방법 - Recall@K



사용자가 관심있는 아이템(1, 2, 3, 5, 7, 8) 중
추천 시스템이 추천한 아이템 수

$$Recall@5 = \frac{3}{6}$$

1. 평가 방법

2. 위치 기반 평가(랭킹 기반 평가)

a. MAP(Mean Average Precision)

i. Average Precision(AP)

$$AP@k = \frac{1}{m} \sum_i Precision@i \cdot rel(i)$$

1. $rel(i)$ 는 relevance(관련성)을 의미. 예를 들어서, 0 또는 1의 값을 가짐
2. m 은 전체 아이템 중 사용자가 좋아했던 아이템 수
3. 즉, 매칭된 아이템만 영향력을 주도록! (0이면 곱했을 때 값이 의미 없어지기 때문)
4. @k기준이므로 앞에서 매칭될수록 점수가 높아짐

ii. Mean Average Precision

- AP들의 평균!

1. 평가 방법 - AP

2. 위치 기반 평가(랭킹 기반 평가)

a. MAP(Mean Average Precision)

i. Average Precision(AP)

추천 영화

① 영화1
② 영화2
③ 영화3

사용자1

영화1 영화10

사용자 선호 영화

$$AP@3 = \frac{1}{2} \left(\left(\frac{1}{1} * 1 \right) + \left(\frac{1}{2} * 0 \right) + \left(\frac{1}{3} * 0 \right) \right) = 0.5$$

추천 영화

① 영화1
② 영화2
③ 영화7

사용자2

영화5 영화7

사용자 선호 영화

$$AP@3 = \frac{1}{2} \left(\left(\frac{0}{1} * 0 \right) + \left(\frac{0}{2} * 0 \right) + \left(\frac{1}{3} * 1 \right) \right) = 0.16$$

1. 평가 방법 - MAP

사용자 선호 개수 = 3으로 가정

연관 있는 아이템

연관 없는 아이템

랭킹 순서	추천 영화	Precision@k
1		1/1
2		1/2
3		1/3

$$AP@3 = \frac{1}{3} \left(\frac{1}{1} * 1 + \frac{1}{2} * 0 + \frac{1}{3} * 0 \right) = 0.33$$

랭킹 순서	추천 영화	Precision@k
1		0/1
2		0/2
3		1/3

$$AP@3 = \frac{1}{3} \left(\frac{0}{1} * 0 + \frac{0}{2} * 0 + \frac{1}{3} * 1 \right) = 0.11$$

$$MAP@3 = \frac{1}{2} (0.33 + 0.11) = 0.22$$

1. 평가

어떤 평가를 할 수 있을까요?

2. 위치 기반 평가(랭킹 기반 평가)

a. NDCG(Normalized Discounted Cumulative Gain)

i. 추천 순위에 가중치를 두어 평가하는 지표이며 1에 가까울수록 좋음

ii. MAP는 선호한 아이템이 추천 리스트에 있으면 1 아니면 0으로 구분하였지만, NDCG는 가중치를 순서별로 다르게 적용함

iii. CG(Cumulative Gain) : 순서를 고려하지 않은, 추천한 아이템의 관련성 합 $CG@k = \sum_i^k rel_i$

iv. DCG(Discounted Cumulative Gain) : CG에 순서를 고려하도록 함 $DCG@k = \sum_i^k \frac{rel_i}{\log_2(i+1)}$

v. IDCG : 최선의 추천을 했을 때 받는 DCG $IDCG@k = \sum_i \frac{rel_i^{opt}}{\log_2(i+1)}$

- 가장 베스트일 때의 경우를 뜻함

i. NDCG는 $DCG/IDCG$ $NDCG@k = \frac{DCG}{IDCG}$

ii. 정리하자면, “가장 이상적으로 추천했을 때 대비 랭킹 기반으로 추천이 잘 제공 되었는가?”를 의미합니다.

1. 평가 - NDCG

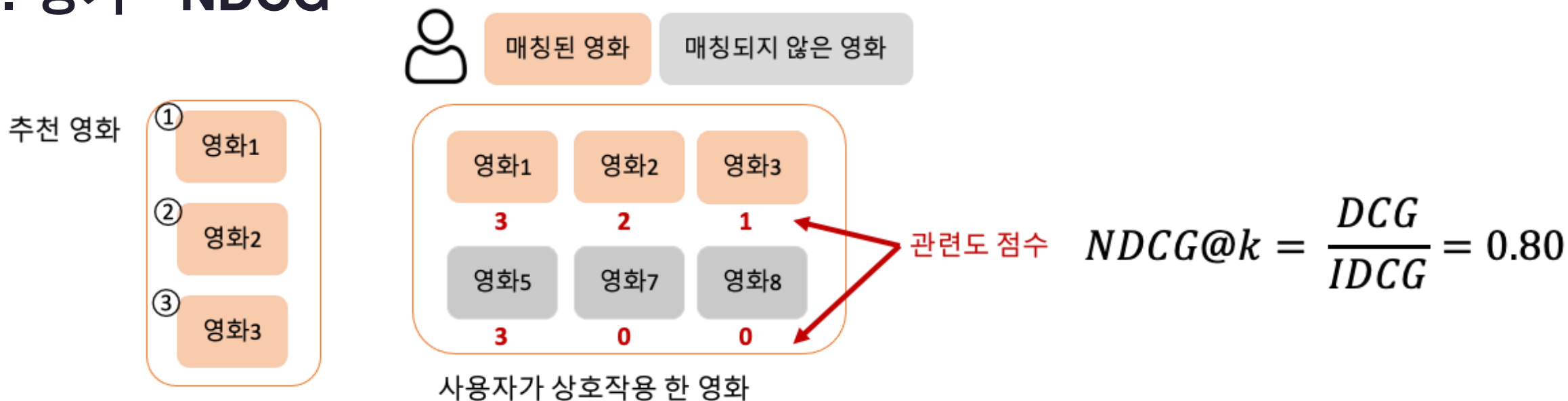
어떤 평가를 할 수 있을까요?

2. 위치 기반 평가(랭킹 기반 평가)

a. NDCG(Normalized Discounted Cumulative Gain)



1. 평가 - NDCG



$$CG@3 = \sum_{i=1}^k rel_i = rel_1 + rel_2 + rel_3 = 3 + 2 + 1 = 6$$

$$DCG@3 = \sum_{i=1}^k \frac{rel_i}{\log_2(i+1)} = \frac{3}{\log_2(1+1)} + \frac{2}{\log_2(2+1)} + \frac{1}{\log_2(3+1)} = 4.76$$

$$IDCG@3 = \sum_{i=1}^k \frac{rel_i^{opt}}{\log_2(i+1)} = \frac{3}{\log_2(1+1)} + \frac{3}{\log_2(2+1)} + \frac{2}{\log_2(3+1)} = 5.89$$

2. 데이터 셋

1. Avazu

- a. 온라인 광고의 CTR 데이터 셋
- b. <https://www.kaggle.com/c/avazu-ctr-prediction>
- c. 사이트 기준
 - i. 사이트 아이디, 도메인, 카테고리
- d. 앱 기준
 - i. 앱 아이디, 도메인, 카테고리
- e. 디바이스 기준
 - i. 디바이스 아이디, IP, 모델, 타입 등
- f. 그 외
 - i. 배너 위치, 익명화된 범주형 데이터 등

Data fields

- id: ad identifier
- click: 0/1 for non-click/click
- hour: format is YYMMDDHH, so 14091123 means 23:00 on Sept. 11, 2014 UTC.
- C1 -- anonymized categorical variable
- banner_pos
- site_id
- site_domain
- site_category
- app_id
- app_domain
- app_category
- device_id
- device_ip
- device_model
- device_type
- device_conn_type
- C14-C21 -- anonymized categorical variables

2. 데이터 셋

2. Criteo

- a. Criteo Labs에서 공개한 온라인 광고 데이터
- b. 4,500만명의 사용자 클릭 히스토리가 존재
- c. 13개의 수치(Numerical)형 필드와 26개의 범주형(Categorical)형 필드가 존재
- d. 광고 클릭률을 예측하는 모델을 개발할 수 있도록 공개된 데이터
- e. <https://www.kaggle.com/c/criteo-display-ad-challenge/overview>

2. 데이터 셋

3. KDD12

- a. 텐센트(Tencent, 중국 포털 서비스)에서 광고 클릭률 예측을 위해 공개한 데이터 셋
- b. KDD Cup 2012, Track2로도 많이 알려져 있으며 KDD Cup에서 주최한 대회를 위해 제공
 - i. <https://www.kaggle.com/c/kddcup2012-track2/overview>
- c. 사용자 인구통계학 정보, 행동 패턴 등이 포함되어 있으며 익명화 처리가 되어있음
- d. 광고 정보에는 광고의 종류, 광고주, 상품 등이 포함되어 있음
- e. 그 외 광고가 표시된 웹사이트 등 정보, 검색 쿼리 정보가 제공됨

2. 데이터 셋

4. MovieLens

- a. 미네소타 대학교의 GroupLens 연구팀이 공개한 영화 추천 시스템 연구에 주로 사용되는 데이터 셋
 - i. <https://grouplens.org/datasets/movielens/>
- b. 사용자, 영화, 평점 정보 등 다양한 정보가 공개되어 있음
- c. 크기에 따라 버전이 나뉘어져 있음
 - i. 100K, 1M, 10M, 20M 등
- d. 본 강의에서 사용할 데이터

02

NCF

1. 논문 리뷰

2. 코드 리뷰

1. 논문 리뷰

Neural Collaborative Filtering

Xiangnan He
National University of
Singapore, Singapore
xiangnanhe@gmail.com

Liqiang Nie
Shandong University
China
nieliqiang@gmail.com

Lizi Liao
National University of
Singapore, Singapore
liaolizi.llz@gmail.com

Xia Hu
Texas A&M University
USA
hu@cse.tamu.edu

Hanwang Zhang
Columbia University
USA
hanwangzhang@gmail.com

Tat-Seng Chua
National University of
Singapore, Singapore
dcscts@nus.edu.sg

출처: X. He, [arxiv NCF\(2017\)](#)

‘다양한 딥러닝 기반 추천 시스템: 특징과 목표’에서

잠깐 살펴봤던 논문입니다.

기억나시나요??

이번에 자세히 살펴보겠습니다.

1. 논문 리뷰

Neural Collaborative Filtering(NCF)

ABSTRACT

In recent years, deep neural networks have yielded immense success on speech recognition, computer vision and natural language processing. However, the exploration of deep neural networks on recommender systems has received relatively less scrutiny. In this work, we strive to develop techniques based on neural networks to tackle the key problem in recommendation — collaborative filtering — on the basis of implicit feedback.

Although some recent work has employed deep learning for recommendation, they primarily used it to model auxiliary information, such as textual descriptions of items and acoustic features of musics. When it comes to model the key factor in collaborative filtering — the interaction between user and item features, they still resorted to matrix factorization and applied an inner product on the latent features of users and items.

출처: X. He, [arxiv NCF\(2017\)](#)

본 논문의 주요 기여사항

1. Collaborative Filtering(CF)에 Neural Network를 적용
2. 암묵적 피드백(Implicit Feedback)을 활용
3. 행렬 분해(Matrix Factorization)는 NCF의 스페셜 케이스
4. 사용자-아이템 상호작용을 학습하기 위해 MLP 구조를 도입함

- MLP : Multi Layer Perceptron

By replacing the inner product with a neural architecture that can learn an arbitrary function from data, we present a general framework named NCF, short for *Neural network-based Collaborative Filtering*. NCF is generic and can express and generalize matrix factorization under its framework. To supercharge NCF modelling with non-linearities, we propose to leverage a multi-layer perceptron to learn the user-item interaction function. Extensive experiments on two real-world datasets show significant improvements of our proposed NCF framework over the state-of-the-art methods. Empirical evidence shows that using deeper layers of neural networks offers better recommendation performance.

1. 논문 리뷰

This work addresses the aforementioned research problems by formalizing a neural network modelling approach for collaborative filtering. We focus on implicit feedback, which indirectly reflects users' preference through behaviours like watching videos, purchasing products and clicking items. Compared to explicit feedback (*i.e.*, ratings and reviews), implicit feedback can be tracked automatically and is thus much easier to collect for content providers. However, it is more challenging to utilize, since user satisfaction is not observed and there is a natural scarcity of negative feedback. In this paper, we explore the central theme of how to utilize DNNs to model noisy implicit feedback signals.

The main contributions of this work are as follows.

1. We present a neural network architecture to model latent features of users and items and devise a general framework NCF for collaborative filtering based on neural networks.
2. We show that MF can be interpreted as a specialization of NCF and utilize a multi-layer perceptron to endow NCF modelling with a high level of non-linearities.
3. We perform extensive experiments on two real-world datasets to demonstrate the effectiveness of our NCF approaches and the promise of deep learning for collaborative filtering.

- CF에 Neural Network를 적용(Implicit feedback)
 - 암묵적(Implicit) 피드백 : 클릭, 구매 등

2.1 Learning from Implicit Data

Let M and N denote the number of users and items, respectively. We define the user-item interaction matrix $\mathbf{Y} \in \mathbb{R}^{M \times N}$ from users' implicit feedback as,

$$y_{ui} = \begin{cases} 1, & \text{if interaction (user } u, \text{ item } i) \text{ is observed;} \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

- Neural Network 기반의 CF인 NCF를 제안
- 행렬 분해는 NCF의 스페셜 케이스
- 기존 모델 대비 좋은 성능

출처: X. He, [arxiv_NCF\(2017\)](#)

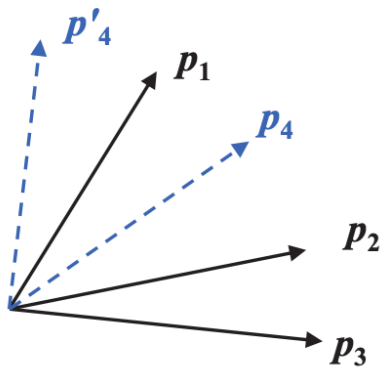
1. 논문 리뷰

	i_1	i_2	i_3	i_4	i_5
u_1	1	1	1	0	1
u_2	0	1	1	0	0
u_3	0	1	1	1	0
u_4	1	0	1	1	1

← items →

↑ users

(a) user-item matrix



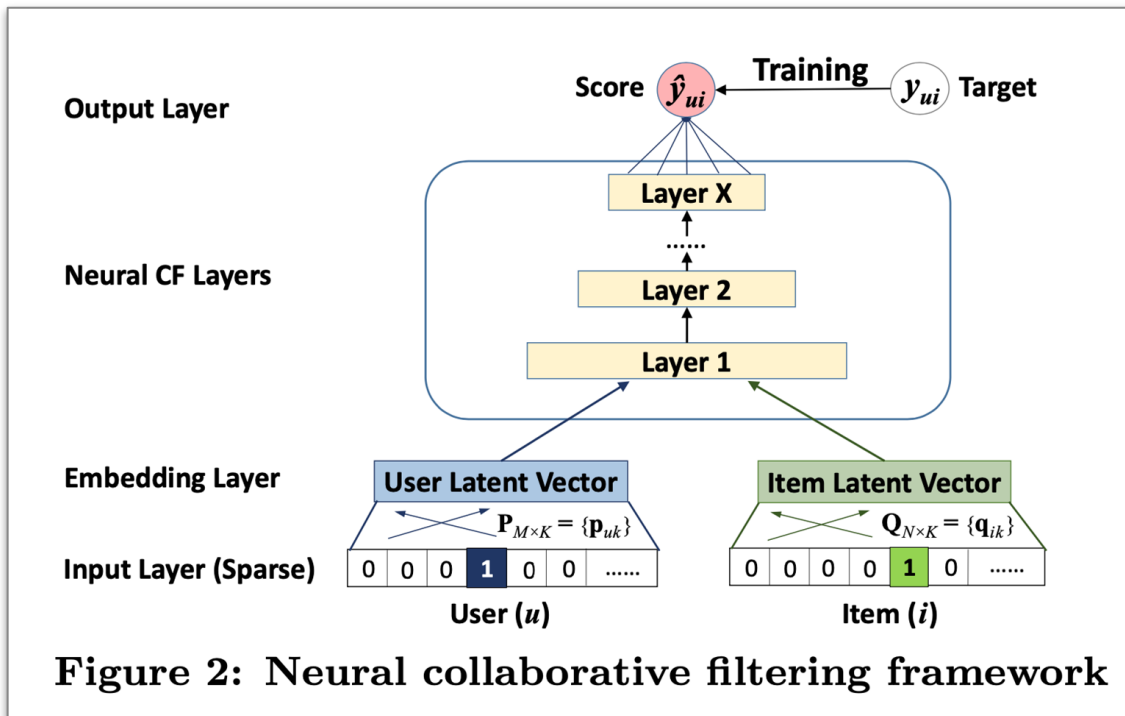
(b) user latent space

Figure 1: An example illustrates MF's limitation. From data matrix (a), u_4 is most similar to u_1 , followed by u_3 , and lastly u_2 . However in the latent space (b), placing p_4 closest to p_1 makes p_4 closer to p_2 than p_3 , incurring a large ranking loss.

출처: X. He, arxiv NCF(2017)

- 기존 선형(linear)한 방법, 단순 내적 방법들의 단점
 - Matrix Factorization(MF), Collaborative Filtering(CF)
- (a)에서 보면
 - u_2, u_3 이 가장 높은 유사도(0.66)
 - 그 다음은 u_1 과 u_2 (0.5) 그리고 u_1 과 u_3 (0.4)
- 이를 (b)에서 표현하면 실선
 - u_2 와 u_3 이 가깝고 그 다음은 u_2 와 u_1
- 만약, 새로운 u_4 가 들어오면?
 - u_4 와 u_1 (0.6), u_4 와 u_3 (0.4), u_4 와 u_2 (0.2)로 가까움
 - 그러나, 이를 잠재 공간에 표현하려면..?
 - 표현이 안된다!!!!
 - 이를 위해 DNN 구조 도입

1. 논문 리뷰



- Figure 2는 CF에 multi-layer를 쌓은 표현임
- 수식으로 표현하면 아래와 같음

$$\hat{y}_{ui} = f(\mathbf{P}^T \mathbf{v}_u^U, \mathbf{Q}^T \mathbf{v}_i^I | \mathbf{P}, \mathbf{Q}, \Theta_f), \quad (3)$$

$$f(\mathbf{P}^T \mathbf{v}_u^U, \mathbf{Q}^T \mathbf{v}_i^I) = \phi_{out}(\phi_X(\dots \phi_2(\phi_1(\mathbf{P}^T \mathbf{v}_u^U, \mathbf{Q}^T \mathbf{v}_i^I))\dots)), \quad (4)$$

$$P \in \mathbb{R}^{m \times k}, Q \in \mathbb{R}^{n \times k} \quad \begin{aligned} M &= \text{number of user,} \\ N &= \text{number of item} \end{aligned}$$

출처: X. He, arxiv_NCF(2017)

1. 논문 리뷰

3.2 Generalized Matrix Factorization (GMF)

We now show how MF can be interpreted as a special case of our NCF framework. As MF is the most popular model for recommendation and has been investigated extensively in literature, being able to recover it allows NCF to mimic a large family of factorization models [26].

Due to the one-hot encoding of user (item) ID of the input layer, the obtained embedding vector can be seen as the latent vector of user (item). Let the user latent vector $\mathbf{p}_u$ be $\mathbf{P}^T \mathbf{v}_u^U$ and item latent vector $\mathbf{q}_i$ be $\mathbf{Q}^T \mathbf{v}_i^I$. We define the mapping function of the first neural CF layer as

$$\phi_1(\mathbf{p}_u, \mathbf{q}_i) = \mathbf{p}_u \odot \mathbf{q}_i, \quad (8)$$

where $\odot$ denotes the element-wise product of vectors. We then project the vector to the output layer:

$$\hat{y}_{ui} = a_{out}(\mathbf{h}^T(\mathbf{p}_u \odot \mathbf{q}_i)), \quad (9)$$

출처: X. He, [arxiv_NCF\(2017\)](#)

- 행렬 분해(Matrix Factorization)는 NCF의 스페셜 케이스
- embedding vector를 사용하면 이게 잠재 벡터
 - 사용자(아이템)를 고정된 저차원 벡터로 매핑하기 때문
- 사용자 잠재 벡터와 아이템 잠재 벡터를 성분별 곱 (Element-wise product)하면 GMF의 결과
 - Element-wise product: 각 행렬의 원소끼리 곱하는 것

1. 논문 리뷰

3.4 Fusion of GMF and MLP

So far we have developed two instantiations of NCF — GMF that applies a linear kernel to model the latent feature interactions, and MLP that uses a non-linear kernel to learn the interaction function from data. The question then arises:

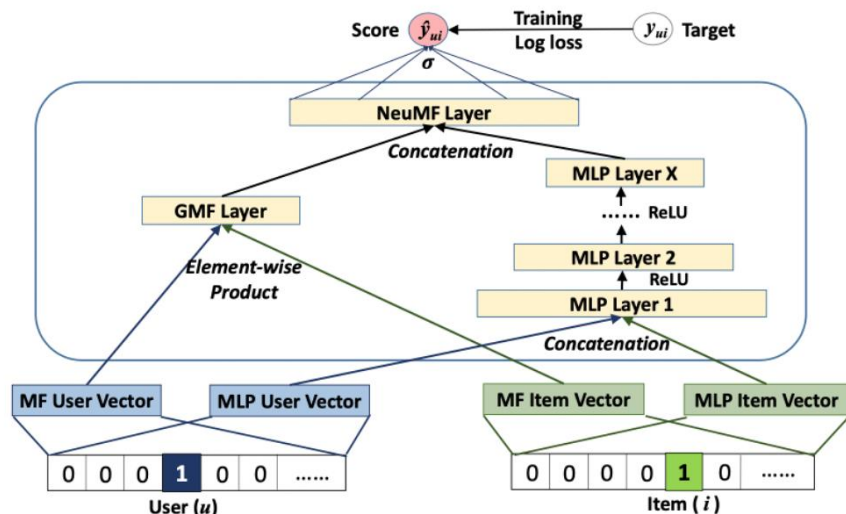


Figure 3: Neural matrix factorization model

출처: X. He, arxiv NCF(2017)

- GMF와 MLP 구조를 합쳐서 최종적으로 모델 활용
- GMF용 Embedding과 MLP용 Embedding을 분리하여 사용함
 - 둘의 역할이 다를 수 있기 때문에 유연성 제공
 - 예) 임베딩 크기를 다르게 한다던가 등
 - 아래 수식처럼 결합

$$\phi^{GMF} = \mathbf{p}_u^G \odot \mathbf{q}_i^G,$$

$$\phi^{MLP} = a_L(\mathbf{W}_L^T(a_{L-1}(\dots a_2(\mathbf{W}_2^T \begin{bmatrix} \mathbf{p}_u^M \\ \mathbf{q}_i^M \end{bmatrix} + \mathbf{b}_2)\dots)) + \mathbf{b}_L),$$

$$\hat{y}_{ui} = \sigma(\mathbf{h}^T \begin{bmatrix} \phi^{GMF} \\ \phi^{MLP} \end{bmatrix}),$$

(12)

p_u^G = user embedding of GMF p_u^M = user embedding of MLP part

q_i^G = item embedding of GMF q_i^M = item embedding of MLP part

1. 논문 리뷰

Table 1: Statistics of the evaluation datasets.

Dataset	Interaction#	Item#	User#	Sparsity
MovieLens	1,000,209	3,706	6,040	95.53%
Pinterest	1,500,809	9,916	55,187	99.73%

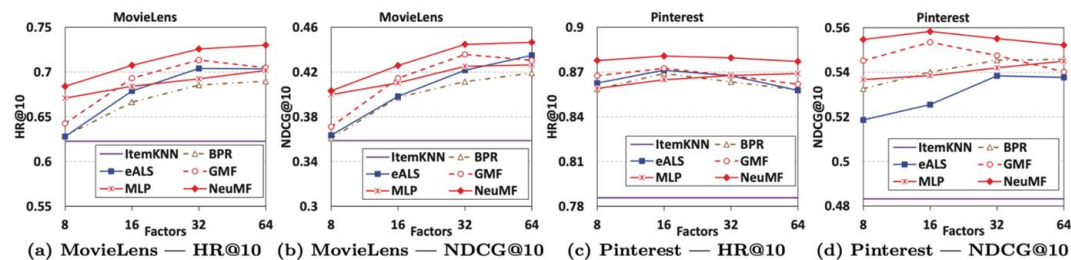


Figure 4: Performance of HR@10 and NDCG@10 w.r.t. the number of predictive factors on the two datasets.

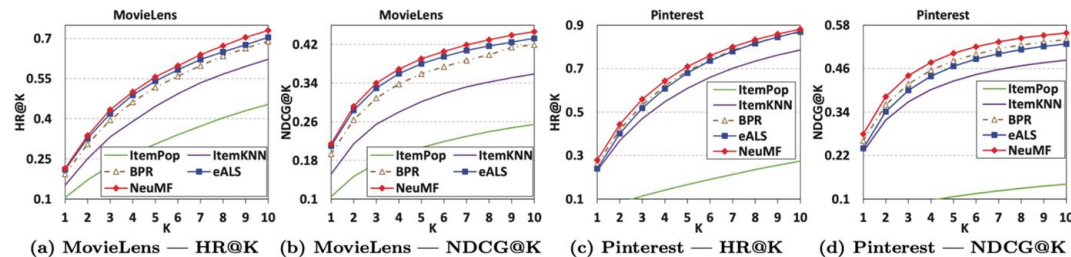


Figure 5: Evaluation of Top-K item recommendation where K ranges from 1 to 10 on the two datasets.

출처: X. He, arxiv NCF(2017)

- 좋은 성능을 보여줌
- NDCG@k : Top@k 중 랭킹 기반 평가
(추천 순위에 가중치를 두어 평가하는 지표. 1에 가까울수록 좋음)
- HitRate@k : Top@k 중 추천 아이템이 들어와 있는 지 매칭 여부

2. 코드 리뷰

```

class NeuMF(torch.nn.Module):
    def __init__(self, config):
        super(NeuMF, self).__init__()
        self.config = config
        self.num_users = config['num_users']
        self.num_items = config['num_items']
        self.latent_dim_mf = config['latent_dim_mf']
        self.latent_dim_mlp = config['latent_dim_mlp']

        self.embedding_user_mlp = torch.nn.Embedding(num_embeddings=self.num_users, embedding_dim=self.latent_dim_mlp)
        self.embedding_item_mlp = torch.nn.Embedding(num_embeddings=self.num_items, embedding_dim=self.latent_dim_mlp)
        self.embedding_user_mf = torch.nn.Embedding(num_embeddings=self.num_users, embedding_dim=self.latent_dim_mf)
        self.embedding_item_mf = torch.nn.Embedding(num_embeddings=self.num_items, embedding_dim=self.latent_dim_mf)

        self.fc_layers = torch.nn.ModuleList()
        for idx, (in_size, out_size) in enumerate(zip(config['layers'][:-1], config['layers'][1:])):
            self.fc_layers.append(torch.nn.Linear(in_size, out_size))

        self.affine_output = torch.nn.Linear(in_features=config['layers'][-1] + config['latent_dim_mf'], out_features=1)
        self.logistic = torch.nn.Sigmoid()

```

1

p_u^G = user embedding of GMF p_u^M = user embedding of MLP part
 q_i^G = item embedding of GMF q_i^M = item embedding of MLP part

1

2

$$\phi^{GMF} = p_u^G \odot q_i^G,$$

$$\phi^{MLP} = a_L(\mathbf{W}_L^T(a_{L-1}(\dots a_2(\mathbf{W}_2^T \begin{bmatrix} p_u^M \\ q_i^M \end{bmatrix} + \mathbf{b}_2)\dots)) + \mathbf{b}_L),$$

$$\hat{y}_{ui} = \sigma(\mathbf{h}^T \begin{bmatrix} \phi^{GMF} \\ \phi^{MLP} \end{bmatrix}),$$

(12)

3

p_u^G = user embedding of GMF p_u^M = user embedding of MLP part
 q_i^G = item embedding of GMF q_i^M = item embedding of MLP part

3

출처 : NCF github

2. 코드 리뷰

```
def forward(self, user_indices, item_indices):
    user_embedding_mlp = self.embedding_user_mlp(user_indices)
    item_embedding_mlp = self.embedding_item_mlp(item_indices)
    user_embedding_mf = self.embedding_user_mf(user_indices)
    item_embedding_mf = self.embedding_item_mf(item_indices)
```

$$\phi^{MLP} = a_L(\mathbf{W}_L^T(a_{L-1}(\dots a_2(\mathbf{W}_2^T \begin{bmatrix} \mathbf{p}_u^M \\ \mathbf{q}_i^M \end{bmatrix} + \mathbf{b}_2)\dots)) + \mathbf{b}_L),$$

```
mlp_vector = torch.cat([user_embedding_mlp, item_embedding_mlp], dim=-1) # the concat latent vector
```

```
mf_vector = torch.mul(user_embedding_mf, item_embedding_mf)
```

```
for idx, _ in enumerate(range(len(self.fc_layers))):
    mlp_vector = self.fc_layers[idx](mlp_vector)
    mlp_vector = torch.nn.ReLU()(mlp_vector)
```

$$\phi_1(\mathbf{p}_u, \mathbf{q}_i) = \mathbf{p}_u \odot \mathbf{q}_i, \quad (8)$$

$$\phi^{MLP} = a_L(\mathbf{W}_L^T(a_{L-1}(\dots a_2(\mathbf{W}_2^T \begin{bmatrix} \mathbf{p}_u^M \\ \mathbf{q}_i^M \end{bmatrix} + \mathbf{b}_2)\dots)) + \mathbf{b}_L),$$

```
vector = torch.cat([mlp_vector, mf_vector], dim=-1)
logits = self.affine_output(vector)
rating = self.logistic(logits)
return rating
```

$$\hat{y}_{ui} = \sigma(\mathbf{h}^T \begin{bmatrix} \phi^{GMF} \\ \phi^{MLP} \end{bmatrix}), \quad (12)$$

출처 : NCF github

```
class NeuMF(tf.keras.Model):
    def __init__(self, config):
        super(NeuMF, self).__init__()
        self.num_users = config['num_users']
        self.num_items = config['num_items']
        self.latent_dim_mf = config['latent_dim_mf']
        self.latent_dim_mlp = config['latent_dim_mlp']

        self.embedding_user_mlp = tf.keras.layers.Embedding(input_dim=self.num_users, output_dim=self.latent_dim_mlp)
        self.embedding_item_mlp = tf.keras.layers.Embedding(input_dim=self.num_items, output_dim=self.latent_dim_mlp)
        self.embedding_user_mf = tf.keras.layers.Embedding(input_dim=self.num_users, output_dim=self.latent_dim_mf)
        self.embedding_item_mf = tf.keras.layers.Embedding(input_dim=self.num_items, output_dim=self.latent_dim_mf)

        self.fc_layers = []
        for idx, (in_size, out_size) in enumerate(zip(config['layers'][:-1], config['layers'][1:])):
            self.fc_layers.append(tf.keras.layers.Dense(out_size, activation='relu'))

        self.affine_output = tf.keras.layers.Dense(units=1, activation='sigmoid')

    def call(self, inputs, training=False):
        user_indices, item_indices = inputs
        user_embedding_mlp = self.embedding_user_mlp(user_indices)
        item_embedding_mlp = self.embedding_item_mlp(item_indices)
        user_embedding_mf = self.embedding_user_mf(user_indices)
        item_embedding_mf = self.embedding_item_mf(item_indices)

        mlp_vector = tf.keras.layers.Concatenate()([user_embedding_mlp, item_embedding_mlp])
        mf_vector = tf.multiply(user_embedding_mf, item_embedding_mf)

        for layer in self.fc_layers:
            mlp_vector = layer(mlp_vector)

        vector = tf.keras.layers.Concatenate()([mlp_vector, mf_vector])
        rating = self.affine_output(vector)
        return rating
```

[출처 : NCF github](#)

2. 코드 리뷰

NCF 리뷰를 해봤습니다.
모델이 복잡하지 않기에,
어렵지 않았을 것이라고 생각합니다.

단순하지만, 강력한 성능을 내주는 NCF이니
잘 배워두시면 좋을 것 같습니다.

03

DeepFM

1. 논문 리뷰
2. 코드 리뷰

1. 논문 리뷰

DeepFM: A Factorization-Machine based Neural Network for CTR Prediction

Huifeng Guo^{*1}, Ruiming Tang², Yunming Ye^{†1}, Zhenguo Li², Xiuqiang He²

¹Shenzhen Graduate School, Harbin Institute of Technology, China

²Noah's Ark Research Lab, Huawei, China

¹huifengguo@yeah.net, yeyunming@hit.edu.cn

²{tangruiming, li.zhenguo, hexiuqiang}@huawei.com

[출처 : H. Huo, IJCAI DeepFM\(2017\)](#)

NCF와 마찬가지로

‘다양한 딥러닝 기반 추천 시스템: 특징과 목표’에서

잠깐 살펴봤던 논문입니다.

역시 자세히 살펴보겠습니다.

1. 논문 리뷰

Abstract

Learning sophisticated feature interactions behind user behaviors is critical in maximizing CTR for recommender systems. Despite great progress, existing methods seem to have a strong bias towards low- or high-order interactions, or require expert feature engineering. In this paper, we show that it is possible to derive an end-to-end learning model that emphasizes both low- and high-order feature interactions. The proposed model, DeepFM, combines the power of factorization machines for recommendation and deep learning for feature learning in a new neural network architecture. Compared to the latest Wide & Deep model from Google, DeepFM has a shared input to its “wide” and “deep” parts, with no need of feature engineering besides raw features. Comprehensive experiments are conducted to demonstrate the effectiveness and efficiency of DeepFM over the existing models for CTR prediction, on both benchmark data and commercial data.

출처 : H. Huo, IJCAI DeepFM(2017)

본 논문의 주요 기여사항

1. 기존 연구들은 Low order 또는 High order에 지나치게 편중되어 있음
2. DeepFM을 제안
 - a. low order와 high order feature interaction을 둘 다 고려
 - b. 두 개의 장점을 결합하고 강조함
3. Factorization Machine(FM)과 Neural network 구조를 결합
4. Wide & Deep 구조에서
 - a. Wide Part를 FM으로 대체
 - b. Feature engineering이 필요 없음

1. 논문 리뷰

are also essential for CTR prediction. To model both low- and high-order feature interactions, [Cheng *et al.*, 2016] proposes an interesting hybrid network structure (*Wide & Deep*) that combines a linear (“wide”) model and a deep model. In this model, two different inputs are required for the “wide part” and “deep part”, respectively, and the input of “wide part” still relies on expertise feature engineering.

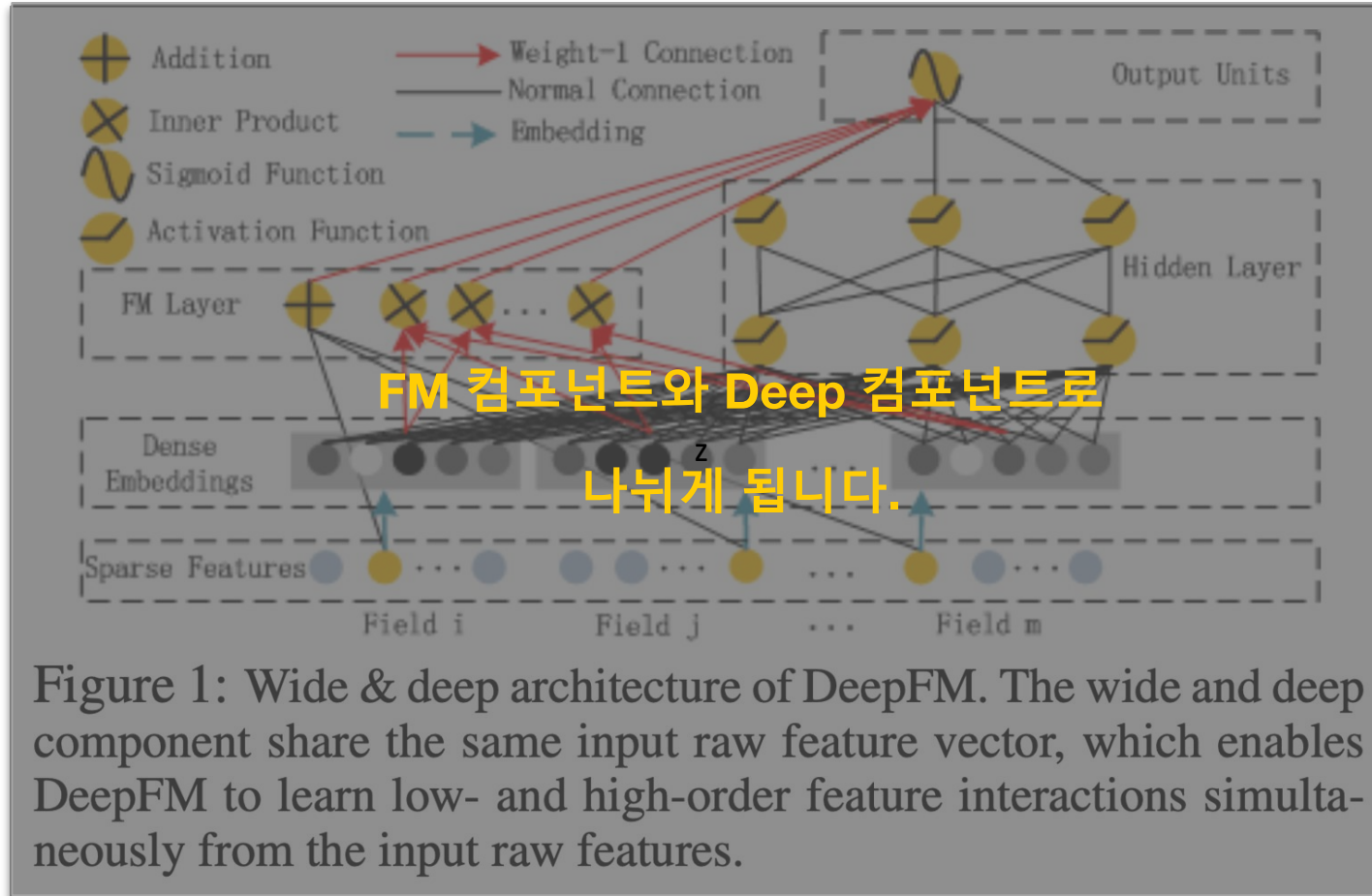
One can see that existing models are biased to low- or high-order feature interaction, or rely on feature engineering. In this paper, we show it is possible to derive a learning model that is able to learn feature interactions of all orders in an end-to-end manner, without any feature engineering besides raw features. Our main contributions are summarized as follows:

출처 : H. Huo, IJCAI DeepFM(2017)

- We propose a new neural network model DeepFM (Figure 1) that integrates the architectures of FM and deep neural networks (DNN). It models low-order feature interactions like FM and models high-order feature interactions like DNN. Unlike the wide & deep model [Cheng *et al.*, 2016], DeepFM can be trained end-to-end without any feature engineering.
- DeepFM can be trained efficiently because its wide part and deep part, unlike [Cheng *et al.*, 2016], share the same input and also the embedding vector. In [Cheng *et al.*, 2016], the input vector can be of huge size as it includes manually designed pairwise feature interactions in the input vector of its wide part, which also greatly increases its complexity.
- We evaluate DeepFM on both benchmark data and commercial data, which shows consistent improvement over existing models for CTR prediction.

- FM과 DNN 구조를 통합한 DeepFM 제안
- FM은 Low order feature interaction을 모델링
- DNN은 High order feature interaction을 모델링
- Feature engineering이 필요가 없음
- Embedding vector를 공유하여 효율적으로 학습
- 기존 연구들보다 좋은 성능이 나옴

1. 논문 리뷰



출처 : H. Huo, IJCAI_DeepFM(2017)

1. 논문 리뷰

FM Component

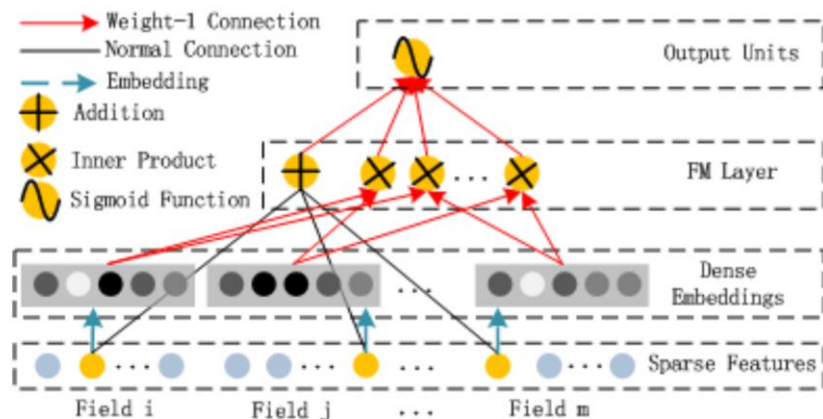


Figure 2: The architecture of FM.

The FM component is a factorization machine, which is proposed in [Rendle, 2010] to learn feature interactions for recommendation. Besides a linear (order-1) interactions among features, FM models pairwise (order-2) feature interactions as inner product of respective feature latent vectors.

- FM 컴포넌트는 Factorization Machine 방법
 - Feature interaction을 학습하기 위해 소개된 방법
- 선형 상호작용(Linear interaction, order-1) 가능
- Pairwise(order-2) feature interaction 가능
 - 내적(Inner product)를 사용

출처 : H. Huo, IJCAI_DeepFM(2017)

1. 논문 리뷰

It can capture order-2 feature interactions much more effectively than previous approaches especially when the dataset is sparse. In previous approaches, the parameter of an interaction of features i and j can be trained only when feature i and feature j both appear in the same data record. While in FM, it is measured via the inner product of their latent vectors V_i and V_j . Thanks to this flexible design, FM can train latent vector V_i (V_j) whenever i (or j) appears in a data record. Therefore, feature interactions, which are never or rarely appeared in the training data, are better learnt by FM.

As Figure 2 shows, the output of FM is the summation of an **Addition** unit and a number of **Inner Product** units:

$$y_{FM} = \langle w, x \rangle + \sum_{j_1=1}^d \sum_{j_2=j_1+1}^d \langle V_{j_1}, V_{j_2} \rangle x_{j_1} \cdot x_{j_2}, \quad (2)$$

where $w \in R^d$ and $V_i \in R^k$ (k is given)². The Addition unit ($\langle w, x \rangle$) reflects the importance of order-1 features, and the Inner Product units represent the impact of order-2 feature interactions.

- 선형 상호작용(Linear interaction, order-1) 가능
- Pairwise(order-2) feature interaction 가능
 - 내적(Inner product)를 사용

출처 : H. Huo, IJCAI_DeepFM(2017)

1. 논문 리뷰

Deep Component

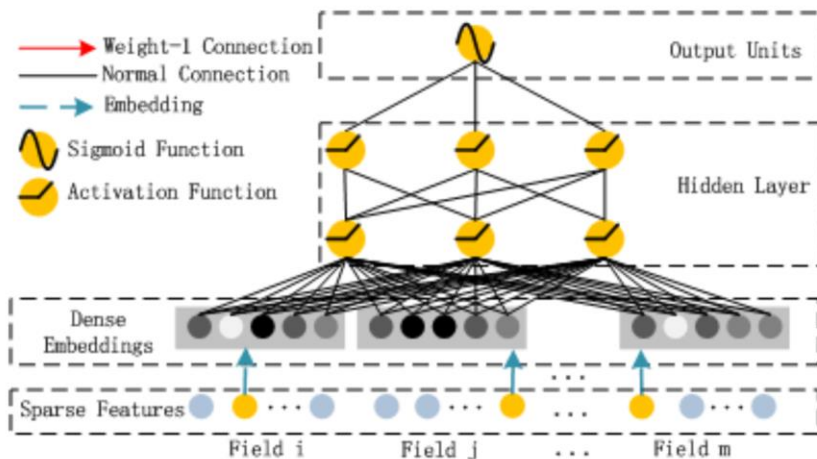


Figure 3: The architecture of DNN.

The deep component is a feed-forward neural network, which is used to learn high-order feature interactions. As shown in Figure 3, a data record (a vector) is fed into the neural network. Compared to neural networks with image [He

- NCF에서 배웠던 MLP쪽 파트와 비슷합니다.
- Deep 컴포넌트는 Feed forward neural network를 사용
- Feature의 임베딩들이 DNN input으로 들어가서 학습되는 것
 - Layer가 중첩되면서 다양한 상호 작용 학습 가능
- FM 컴포넌트에서 사용되는 Feature embedding을 공유함
 - Raw feature에서 Low, High 상호작용을 학습 가능
 - Feature engineering 과정이 필요 없음

$$a^{(0)} = [e_1, e_2, \dots, e_m], \quad (3)$$

$$a^{(l+1)} = \sigma(W^{(l)} a^{(l)} + b^{(l)}), \quad (4)$$

$$y_{DNN} = \sigma(W^{|H|+1} \cdot a^H + b^{|H|+1}),$$

출처 : H. Huo, IJCAI DeepFM(2017)

1. 논문 리뷰

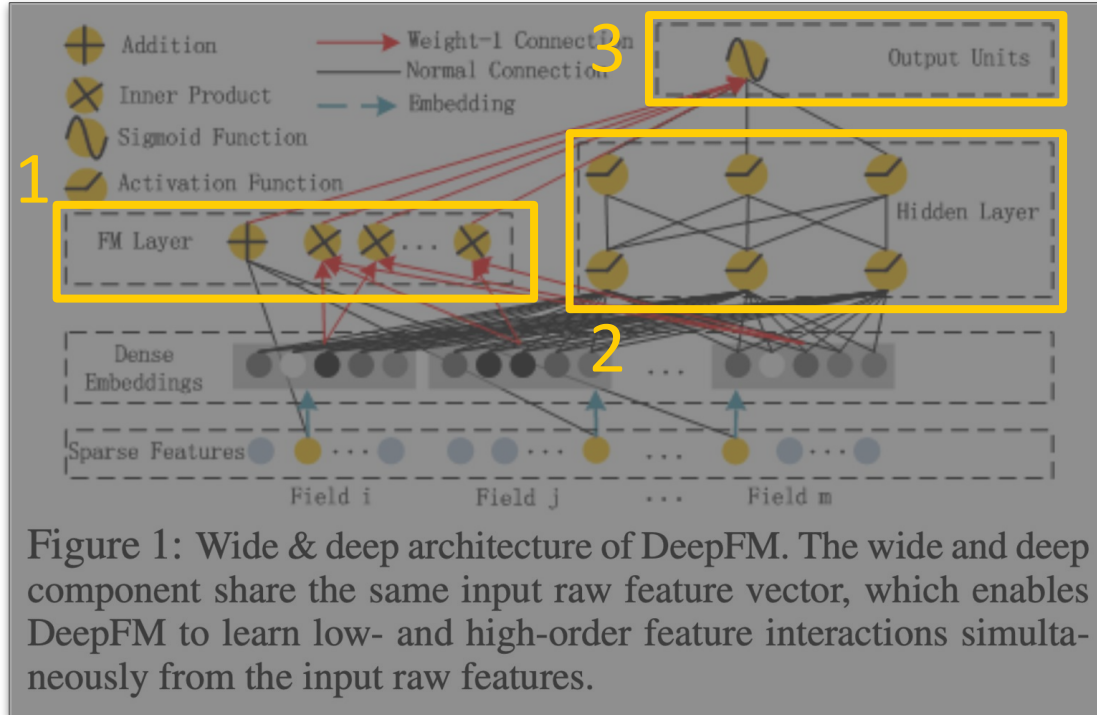


Figure 1: Wide & deep architecture of DeepFM. The wide and deep component share the same input raw feature vector, which enables DeepFM to learn low- and high-order feature interactions simultaneously from the input raw features.

출처 : H. Huo, IJCAI_DeepFM(2017)

2.1 DeepFM

We aim to learn both low- and high-order feature interactions. To this end, we propose a Factorization-Machine based neural network (DeepFM). As depicted in Figure 1¹, DeepFM consists of two components, *FM component* and *deep component*, that share the same input. For feature i , a scalar w_i is used to weigh its order-1 importance, a latent vector V_i is used to measure its impact of interactions with other features. V_i is fed in FM component to model order-2 feature interactions, and fed in deep component to model high-order feature interactions. ² All parameters, including w_i , V_i , and the network parameters ($W^{(l)}$, $b^{(l)}$ below) are trained jointly for the combined prediction model: ³

$$\hat{y} = \text{sigmoid}(y_{FM} + y_{DNN}), \quad (1)$$

where $\hat{y} \in (0, 1)$ is the predicted CTR, y_{FM} is the output of FM component, and y_{DNN} is the output of deep component.

1. 논문 리뷰

Table 2: Performance on CTR prediction.

	Company*		Criteo	
	AUC	LogLoss	AUC	LogLoss
LR	0.8640	0.02648	0.7686	0.47762
FM	0.8678	0.02633	0.7892	0.46077
FNN	0.8683	0.02629	0.7963	0.45738
IPNN	0.8664	0.02637	0.7972	0.45323
OPNN	0.8658	0.02641	0.7982	0.45256
PNN*	0.8672	0.02636	0.7987	0.45214
LR & DNN	0.8673	0.02634	0.7981	0.46772
FM & DNN	0.8661	0.02640	0.7850	0.45382
DeepFM	0.8715	0.02618	0.8007	0.45083

Overall, our proposed DeepFM model beats the competitors by more than 0.37% and 0.42% in terms of AUC and Logloss on Company* dataset, respectively. In fact, a small improvement in offline AUC evaluation is likely to lead to a significant increase in online CTR. As reported in [Cheng *et al.*, 2016], compared with LR, Wide & Deep improves AUC by 0.275% (offline) and the improvement of online CTR is 3.9%. The daily turnover of Company*'s App Store is millions of dollars, therefore even several percents lift in CTR brings extra millions of dollars each year.

출처 : H. Huo, IJCAI_DeepFM(2017)

1. 논문 리뷰

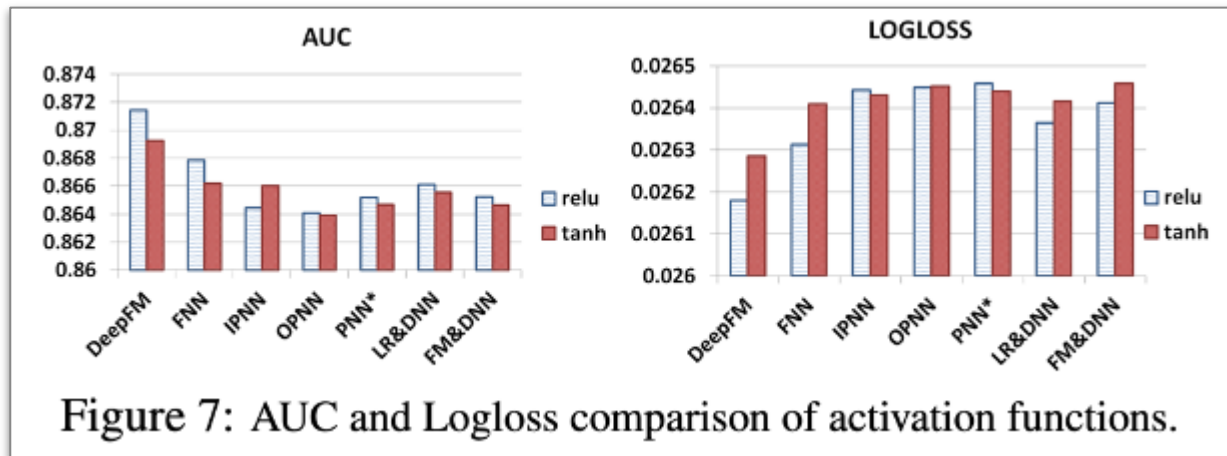


Figure 7: AUC and Logloss comparison of activation functions.

출처 : H. Huo, IJCAI DeepFM(2017)

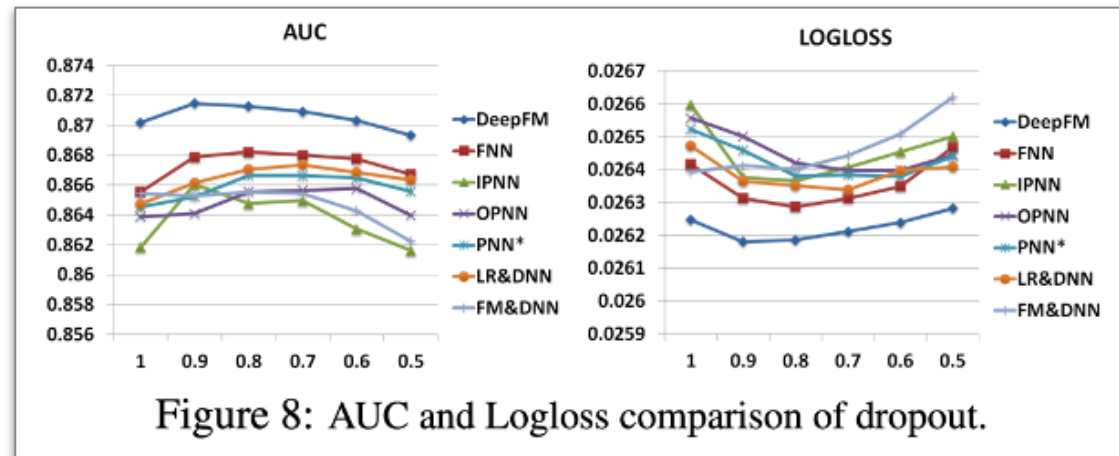


Figure 8: AUC and Logloss comparison of dropout.

1. 논문 리뷰

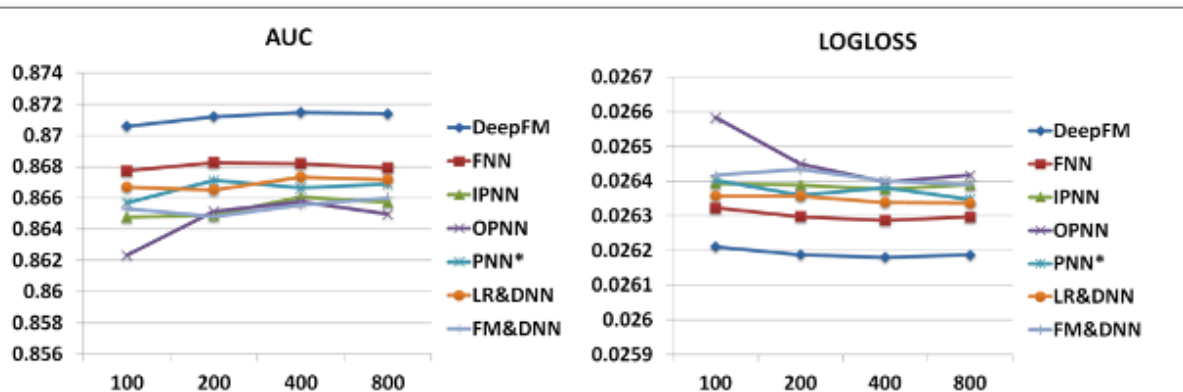


Figure 9: AUC and Logloss comparison of number of neurons.

출처 : H. Huo, IJCAI DeepFM(2017)

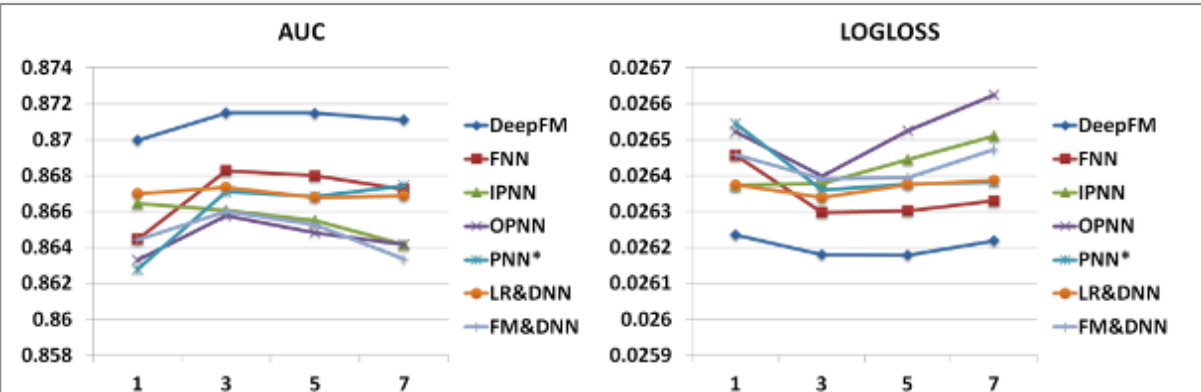


Figure 10: AUC and Logloss comparison of number of layers.

2. 코드 리뷰

FeatureEmbedding

```
class FeaturesEmbedding(torch.nn.Module):  
  
    def __init__(self, field_dims, embed_dim):  
        super().__init__()  
        self.embedding = torch.nn.Embedding(sum(field_dims), embed_dim)  
        self.offsets = np.array((0, *np.cumsum(field_dims)[: -1]), dtype=np.float64)  
        torch.nn.init.xavier_uniform_(self.embedding.weight.data)  
  
    def forward(self, x):  
        x = x + x.new_tensor(self.offsets).unsqueeze(0)  
        return self.embedding(x)
```

출처 :DeepFM github

- Feature에 해당되는 Embedding 값을 가지고 옴
- Offsets: 필드로 구분된 Feature가 시작되는 각 지점
- Forward결과 Feature에 해당되는 Embedding에서 Vector 값을 가지고 옴

03 DeepFM

2. 코드 리뷰

FeatureEmbedding

```
class FeaturesEmbedding(torch.nn.Module):  
    def __init__(self, field_dims, embed_dim):  
        super().__init__()  
        self.embedding = torch.nn.Embedding(sum(field_dims), embed_dim)  
        self.offsets = np.array((0, *np.cumsum(field_dims)[:len(field_dims)-1]), dtype=np.float64)  
        torch.nn.init.xavier_uniform_(self.embedding.weight.data)  
  
    def forward(self, x):  
        x = x + x.new_tensor(self.offsets).unsqueeze(0)  
        return self.embedding(x)
```

출처 :DeepFM github

```
field_dims = [10, 20, 30]  
num_samples = 5  
num_features = len(field_dims)  
sample_input = torch.zeros((num_samples, num_features), dtype=torch.long)  
  
for i in range(num_features):  
    sample_input[:, i] = torch.randint(0, field_dims[i], (num_samples,))  
  
embed_dim = 4  
print(sample_input)
```

```
tensor([[ 1, 18, 12],  
        [ 6, 19,  6],  
        [ 4, 18,  6],  
        [ 4, 14, 22],  
        [ 0, 19, 11]])
```

embedding_output = features_embedding(sample_input)

1. x : tensor([[1, 18, 12],
 [6, 19, 6],
 [4, 18, 6],
 [4, 14, 22],
 [0, 19, 11]])

2. x.new_tensor(self.offsets) : tensor([0, 10, 30])

3. x + x.new_tensor(self.offsets).unsqueeze(0) : tensor([[1, 28, 42],
 [6, 29, 36],
 [4, 28, 36],
 [4, 24, 52],
 [0, 29, 41]])

offset만큼 더해집니
다

2. 코드 리뷰

FeatureEmbedding

```
class FeaturesEmbedding(torch.nn.Module):

    def __init__(self, field_dims, embed_dim):
        super().__init__()
        self.embedding = torch.nn.Embedding(sum(field_dims), embed_dim)
        self.offsets = np.array((0, *np.cumsum(field_dims)[: -1]), dtype=np.float64)
        torch.nn.init.xavier_uniform_(self.embedding.weight.data)

    def forward(self, x):
        x = x + x.new_tensor(self.offsets).unsqueeze(0)
        return self.embedding(x)
```

출처 :DeepFM github

```
print("Output from FeaturesEmbedding:", embedding_output)
```

```
Output from FeaturesEmbedding: tensor([[[ 0.1039, -0.1280,  0.1296, -0.1459],
      [-0.1164,  0.1205,  0.0602,  0.0901],
      [ 0.2258, -0.0163,  0.2447,  0.2356]],

    [[-0.0784,  0.1992,  0.0700, -0.2221],
      [-0.0928, -0.2686,  0.0683,  0.1382],
      [ 0.2970,  0.0599,  0.2365,  0.1991]],

    [[-0.0633,  0.0248,  0.1004, -0.0605],
      [ 0.2899,  0.1345, -0.2662,  0.1719],
      [-0.3039,  0.0866, -0.1776, -0.1099]],

    [[ 0.2812,  0.2565, -0.1346, -0.0457],
      [-0.1282,  0.0500,  0.1786, -0.2759],
      [-0.0950,  0.2564, -0.2595,  0.0010]],

    [[ 0.1919, -0.0105,  0.2307, -0.2841],
      [ 0.1752, -0.1240, -0.1311, -0.1162],
      [-0.2839, -0.1828,  0.0401, -0.0190]]], grad_fn=<EmbeddingBackward0>)
```

2. 코드 리뷰

FeaturesLinear

```
class FeaturesLinear(torch.nn.Module):  
  
    def __init__(self, field_dims, output_dim=1):  
        super().__init__()  
        self.fc = torch.nn.Embedding(sum(field_dims), output_dim)  
        self.bias = torch.nn.Parameter(torch.zeros((output_dim,)))  
        self.offsets = np.array((0, *np.cumsum(field_dims)[:len(field_dims)-1]), dtype=np.float64)  
  
    def forward(self, x):  
        x = x + x.new_tensor(self.offsets).unsqueeze(0)  
        return torch.sum(self.fc(x), dim=1) + self.bias
```

[출처 :DeepFM github](#)

- order-1인 Linear 연산을 구하는 역할 수행
- Output dim은 1차원(스칼라 값)

03 DeepFM

2. 코드 리뷰

FeaturesLinear

```
class FeaturesLinear(torch.nn.Module):  
    def __init__(self, field_dims, output_dim=1):  
        super().__init__()  
        self.fc = torch.nn.Embedding(sum(field_dims), output_dim)  
        self.bias = torch.nn.Parameter(torch.zeros((output_dim,)))  
        self.offsets = np.array((0, *np.cumsum(field_dims)[::-1]), dtype=np.float64)  
  
    def forward(self, x):  
        x = x + x.new_tensor(self.offsets).unsqueeze(0)  
        return torch.sum(self.fc(x), dim=1) + self.bias
```

[출처 :DeepFM github](#)

- order-1인 Linear 연산을 구하는 역할 수행
- Output dim은 1차원(스칼라 값)

```
# Forward pass through the modules  
linear_output = features_linear(sample_input)
```

```
1. x : tensor([[ 1, 18, 12],  
               [ 6, 19,  6],  
               [ 4, 18,  6],  
               [ 4, 14, 22],  
               [ 0, 19, 11]])  
2. x.new_tensor(self.offsets) : tensor([ 0, 10, 30])  
3. x + x.new_tensor(self.offsets).unsqueeze(0) : tensor([[ 1, 28, 42],  
               [ 6, 29, 36],  
               [ 4, 28, 36],  
               [ 4, 24, 52],  
               [ 0, 29, 41]])
```

```
# Print the outputs  
print("Output from FeaturesLinear:", linear_output)
```

```
Output from FeaturesLinear: tensor([[ -1.4318],  
                                   [ 3.3070],  
                                   [-1.3821],  
                                   [-0.1456],  
                                   [ 1.9126]], grad_fn=<AddBackward0>)
```

2. 코드 리뷰

FactorizationMachine

```
class FactorizationMachine(torch.nn.Module):

    def __init__(self, reduce_sum=True):
        super().__init__()
        self.reduce_sum = reduce_sum

    def forward(self, x):
        square_of_sum = torch.sum(x, dim=1) ** 2
        sum_of_square = torch.sum(x ** 2, dim=1)
        ix = square_of_sum - sum_of_square
        if self.reduce_sum:
            ix = torch.sum(ix, dim=1, keepdim=True)
        return 0.5 * ix
```

출처 : DeepFM github

training data, are better learnt by FM.

As Figure 2 shows, the output of FM is the summation of an **Addition** unit and a number of **Inner Product** units:

$$y_{FM} = \langle w, x \rangle + \sum_{j_1=1}^d \sum_{j_2=j_1+1}^d \langle V_{i_1}, V_{i_2} \rangle x_{j_1} \cdot x_{j_2}, \quad (2)$$

where $w \in R^d$ and $V_i \in R^k$ (k is given)². The Addition unit ($\langle w, x \rangle$) reflects the importance of order-1 features, and the Inner Product units represent the impact of order-2 feature interactions.

출처 : H. Huo, IJCAI DeepFM(2017)

$$\begin{aligned} & \sum_{i=1}^n \sum_{j=i+1}^n \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j \\ &= \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j - \frac{1}{2} \sum_{i=1}^n \langle \mathbf{v}_i, \mathbf{v}_i \rangle x_i x_i \\ &= \frac{1}{2} \left(\sum_{i=1}^n \sum_{j=1}^n \sum_{f=1}^k v_{i,f} v_{j,f} x_i x_j - \sum_{i=1}^n \sum_{f=1}^k v_{i,f} v_{i,f} x_i x_i \right) \\ &= \frac{1}{2} \sum_{f=1}^k \left(\left(\sum_{i=1}^n v_{i,f} x_i \right) \left(\sum_{j=1}^n v_{j,f} x_j \right) - \sum_{i=1}^n v_{i,f}^2 x_i^2 \right) \\ &= \frac{1}{2} \sum_{f=1}^k \left(\left(\sum_{i=1}^n v_{i,f} x_i \right)^2 - \sum_{i=1}^n v_{i,f}^2 x_i^2 \right) \end{aligned}$$

출처 : S. Rendle, IEEE-FM(2010)

1 2

2. 코드 리뷰

MultiLayerPerceptron

```
class MultiLayerPerceptron(torch.nn.Module):

    def __init__(self, input_dim, embed_dims, dropout, output_layer=True):
        super().__init__()
        layers = list()
        for embed_dim in embed_dims:
            layers.append(torch.nn.Linear(input_dim, embed_dim))
            layers.append(torch.nn.BatchNorm1d(embed_dim))
            layers.append(torch.nn.ReLU())
            layers.append(torch.nn.Dropout(p=dropout))
            input_dim = embed_dim
        if output_layer:
            layers.append(torch.nn.Linear(input_dim, 1))
        self.mlp = torch.nn.Sequential(*layers)

    def forward(self, x):
        return self.mlp(x)
```

출처 : [DeepFM github](#)

Deep Component

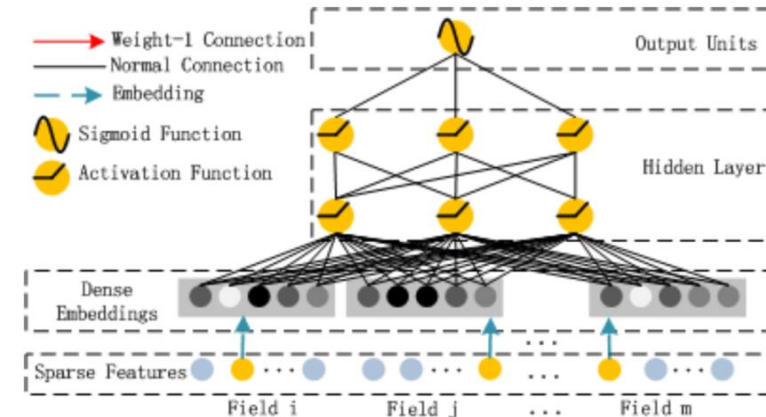


Figure 3: The architecture of DNN.

The deep component is a feed-forward neural network, which is used to learn high-order feature interactions. As shown in Figure 3, a data record (a vector) is fed into the neural network. Compared to neural networks with image [He

$$a^{(0)} = [e_1, e_2, \dots, e_m], \quad (3)$$

$$a^{(l+1)} = \sigma(W^{(l)}a^{(l)} + b^{(l)}), \quad (4)$$

$$y_{DNN} = \sigma(W^{|H|+1} \cdot a^H + b^{|H|+1}),$$

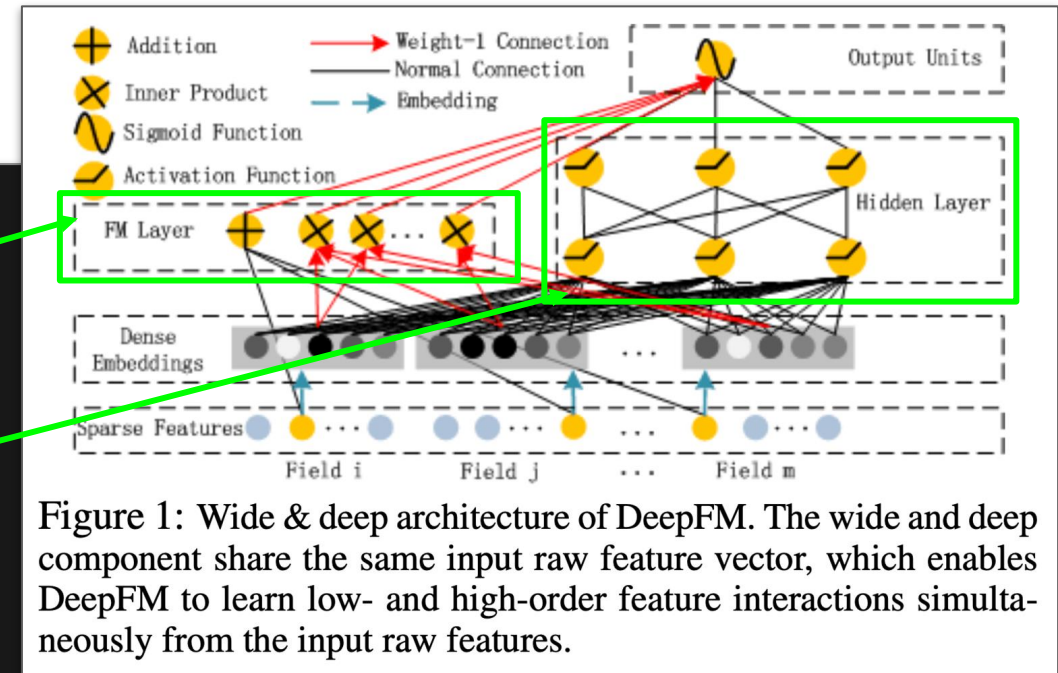
출처 : [H. Huo, IJCAI DeepFM\(2017\)](#)

2. 코드 리뷰

```
class DeepFactorizationMachineModel(torch.nn.Module):

    def __init__(self, field_dims, embed_dim, mlp_dims, dropout):
        super().__init__()
        self.linear = FeaturesLinear(field_dims)
        self.fm = FactorizationMachine(reduce_sum=True)
        self.embedding = FeaturesEmbedding(field_dims, embed_dim)
        self.embed_output_dim = len(field_dims) * embed_dim
        self.mlp = MultiLayerPerceptron(self.embed_output_dim, mlp_dims, dropout)

    def forward(self, x):
        embed_x = self.embedding(x)
        x = self.linear(x) + self.fm(embed_x) + self.mlp(embed_x.view(-1, self.embed_output_dim))
        return torch.sigmoid(x.squeeze(1))
```



출처 : H. Huo, IJCAI_DeepFM(2017)

출처 : DeepFM_github

2. 코드 리뷰

```
class DeepFactorizationMachineModel(torch.nn.Module):
```

```
    def __init__(self, field_dims, embed_dim, mlp_dims, dropout):
```

```
        super().__init__()
```

```
        self.linear = FeaturesLinear(field_dims)
```

```
        self.fm = FactorizationMachine(reduce_sum=True)
```

```
        self.embedding = FeaturesEmbedding(field_dims, embed_dim)
```

```
        self.embed_output_dim = len(field_dims) * embed_dim
```

```
        self.mlp = MultiLayerPerceptron(self.embed_output_dim, mlp_dims, dropout)
```

```
    def forward(self, x):
```

```
        embed_x = self.embedding(x)
```

```
        x = self.linear(x) + self.fm(embed_x) + self.mlp(embed_x.view(-1, self.embed_output_dim))
```

```
        return torch.sigmoid(x.squeeze(1))
```

3

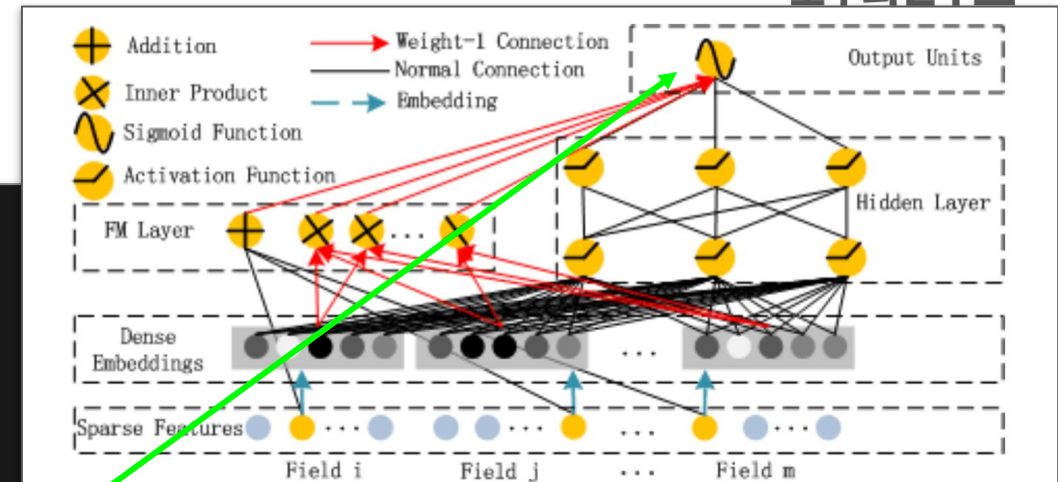


Figure 1: Wide & deep architecture of DeepFM. The wide and deep component share the same input raw feature vector, which enables DeepFM to learn low- and high-order feature interactions simultaneously from the input raw features.

출처 : H. Huo, IJCAI_DeepFM(2017)

출처 :DeepFM_github

2. 코드 리뷰

DeepFM 모델을 리뷰해봤습니다.

NCF보단 조금 어려우시죠?

**DeepFM도 정말 좋은 성능을 보여주는
기본 모델입니다.**

**발표된 2017년부터 지금까지 꾸준히 다양한 방법으로
응용하여 많이 사용되고 있습니다!**

04

AutoInt

1. 논문 리뷰
2. 코드 리뷰

1. 논문 리뷰

AutoInt: Automatic Feature Interaction Learning via Self-Attentive Neural Networks

Weiping Song*
Department of Computer Science,
School of EECS, Peking University
weiping.song@pku.edu.cn

Chence Shi
Department of Computer Science,
School of EECS, Peking University
chenceshi@pku.edu.cn

Zhiping Xiao
Department of Computer Science,
University of California, Los Angeles
patriciaxiao@g.ucla.edu

Zhijian Duan, Yewen Xu
Department of Computer Science,
School of EECS, Peking University
{zjduan,xuyewen}@pku.edu.cn

Ming Zhang†
Department of Computer Science,
School of EECS, Peking University
mzhang_cs@pku.edu.cn

Jian Tang†
Mila-Quebec AI Institute,
HEC Montreal & CIFAR AI Chair
jian.tang@hec.ca

ABSTRACT

Click-through rate (CTR) prediction, which aims to predict the probability of a user clicking on an ad or an item, is critical to many online applications such as online advertising and recommender systems. The problem is very challenging since (1) the input features (e.g., the user id, user age, item id, item category) are usually sparse and high-dimensional, and (2) an effective prediction relies on high-order combinatorial features (a.k.a. cross features), which are very time-consuming to hand-craft by domain experts and are impossible to be enumerated. Therefore, there have been efforts in finding low-dimensional representations of the sparse and high-dimensional raw features and their meaningful combinations.

In this paper, we propose an effective and efficient method called the *AutoInt* to automatically learn the high-order feature interactions of input features. Our proposed algorithm is very general,

CCS CONCEPTS

• Information systems → Recommender systems; • Computing methodologies → Neural networks; Learning latent representations;

KEYWORDS

High-order feature interactions, Self attention, CTR prediction, Explainable recommendation

ACM Reference Format:

Weiping Song, Chence Shi, Zhiping Xiao, Zhijian Duan, Yewen Xu, Ming Zhang, and Jian Tang. 2019. AutoInt: Automatic Feature Interaction Learning via Self-Attentive Neural Networks. In *The 28th ACM International Conference on Information and Knowledge Management (CIKM '19)*, November 3–7, 2019, Beijing, China. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3357384.3357925>

본 논문의 주요 기여사항

1. 입력 데이터들은 희소(Sparse)하고 고차원(High-dimensional)적 데이터
2. 입력 피쳐들 간 고차 상호작용(High-order feature interaction)은 여러 어려움이 있음
 - a. 수작업은 도메인 전문가에 의존하게 되며 모든 조합을 만들기는 불가능
 - b. ML 모델이 학습하도록 할 수 있지만 가능한 모든 High-order feature를 열거하면 차원과 희소성이 급격히 증가하여 모델 과적합 문제 발생
 - c. 희소하고 고차원 특징을 가진 입력을 저차원(Low-dimension)으로 표현하여 매핑해 다양한 조합을 모델링하는 노력이 과거에 있었음

v2 [cs.IR] 23 Aug 2019

출처 : [Song, cikm AutoInt\(2019\)](#)

1. 논문 리뷰

For example, Factorization Machines (FM) [26], which combine polynomial regression models with factorization techniques, are developed to model feature interactions and have been proved effective for various tasks [27, 28]. However, limited by its polynomial fitting time, it is only effective for modeling low-order feature interactions and impractical to capture high-order feature interactions. Recently, many works [8, 11, 13, 38] based on deep neural networks have been proposed to model the high-order feature interactions. Specifically, multiple layers of non-linear neural networks are usually used to capture the high-order feature interactions. However, such kinds of methods suffer from two limitations. First, fully-connected neural networks have been shown inefficient in learning multiplicative feature interactions [4]. Second, since these models learn the feature interactions in an implicit way, they lack good explanation on which feature combinations are meaningful. Therefore, we are looking for an approach that is able to explicitly model different orders of feature combinations, represent the entire features into low-dimensional spaces, and meanwhile offer good model explainability.

출처 : [Song, ckm AutoInt\(2019\)](#)

3. 과거의 노력들

a. FM

- i. 계산 복잡도 한계로 저차원 특징 조합(Low-order feature)에만 효과적(second-order)
- ii. High-order에는 비현실적

b. DNN 기반

- i. 비선형 뉴럴네트워크(Non-linear neural network) 기반
- ii. 앞선 연구에 따르면, 특징 사이의 곱셈 관계(Multiplicative feature interaction)에는 비효율적으로 나왔다고 함
 - i. 암묵적으로 학습하기에 설명력이 부족함
 - 어떤 조합이 유용한지 설명하기가 어려움

1. 논문 리뷰

interactions between different features. Within each interacting layer, each feature is allowed to interact with all the other features and is able to automatically identify relevant features to form meaningful higher-order features via the multi-head attention mechanism [36]. Moreover, the multi-head mechanism projects a feature into multiple subspaces, and hence it can capture different feature interactions in different subspaces. Such an interacting layer models the one-step interaction between the features. By stacking multiple interacting layers, we are able to model different orders of feature interactions. In practice, the residual connection [12] is added to the interacting layer, which allows combining different orders of

feature combinations. We use the attention mechanism for measuring the correlations between features, which offers good model explainability.

To summarize, in this paper we make the following contributions:

- We propose to study the problem of explicitly learning high-order feature interactions and meanwhile finding models with good explainability for the problem.
- We propose a novel approach based on self-attentive neural network, which can automatically learn high-order feature interactions and efficiently handle large-scale high-dimensional sparse data.

4. 본 논문은 AutoInt 모델을 제안

- a. 희소하고 고차원 특징을 가진 입력 데이터를 저차원으로 표현
- b. 멀티헤드 셀프 어텐션(Multi-Head Self-Attention)구조를 활용해 입력 피쳐들간의 상호작용과 관계성을 모델링
- c. 쌓아올린 구조(Stack)를 통해 서로 다른 차원에서 피쳐 상호작용 학습
- d. 고차 상호작용을 명시적(Explicitly) & 자동(Automatic)으로 학습
- e. 좋은 설명 가능성 제공

1. 논문 리뷰

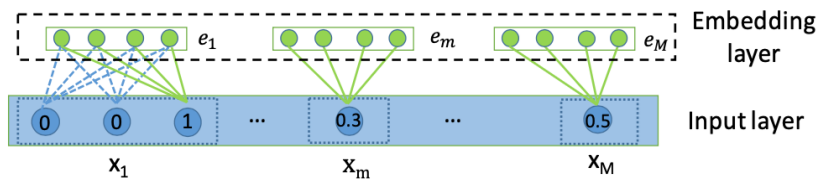


Figure 2: Illustration of input and embedding layer, where both categorical and numerical fields are represented by low-dimensional dense vectors.

4.2 Input Layer

We first represent user's profiles and item's attributes as a sparse vector, which is the concatenation of all fields. Specifically,

$$\mathbf{x} = [\mathbf{x}_1; \mathbf{x}_2; \dots; \mathbf{x}_M], \quad (1)$$

where M is the number of total feature fields, and $\mathbf{x}_i$ is the feature representation of the i -th field. $\mathbf{x}_i$ is a one-hot vector if the i -th field is categorical (e.g., $\mathbf{x}_1$ in Figure 2). $\mathbf{x}_i$ is a scalar value if the i -th field is numerical (e.g., $\mathbf{x}_M$ in Figure 2).

출처 : [Song, ckm AutoInt\(2019\)](#)

제안한 방법 : Input & Embedding layer

Input layer

- 사용자 프로필과 아이템 속성들을 가져옴
- 모든 필드를 전부 이어줌(concat)
- 범주형(categorical) 데이터: x 는 one-hot 형식
- 수치형(numerical) 데이터: scalar value

1. 논문 리뷰

4.3 Embedding Layer

Since the feature representations of the categorical features are very sparse and high-dimensional, a common way is to represent them into low-dimensional spaces (e.g., word embeddings). Specifically, we represent each categorical feature with a low-dimensional vector, i.e.,

$$\mathbf{e}_i = \mathbf{V}_i \mathbf{x}_i, \quad (2)$$

$$\mathbf{e}_m = \mathbf{v}_m x_m, \quad (4)$$

where $\mathbf{v}_m$ is an embedding vector for field m , and x_m is a scalar value.

출처 : [Song, ckm AutoInt\(2019\)](#)

제안한 방법 : Input & Embedding layer

Embedding layer

- 범주형 데이터 : 매우 희소(Sparse)하고 고차원(High-dimensional)
 - 이들을 저차원 벡터로 표현
- 수치형 데이터 : Embedding vector와 곱해주는 방식으로 진행

1. 논문 리뷰

$$\alpha_{m,k}^{(h)} = \frac{\exp(\psi^{(h)}(\mathbf{e}_m, \mathbf{e}_k))}{\sum_{l=1}^M \exp(\psi^{(h)}(\mathbf{e}_m, \mathbf{e}_l))}, \quad (5)$$

$$\psi^{(h)}(\mathbf{e}_m, \mathbf{e}_k) = \langle \mathbf{W}_{\text{Query}}^{(h)} \mathbf{e}_m, \mathbf{W}_{\text{Key}}^{(h)} \mathbf{e}_k \rangle,$$

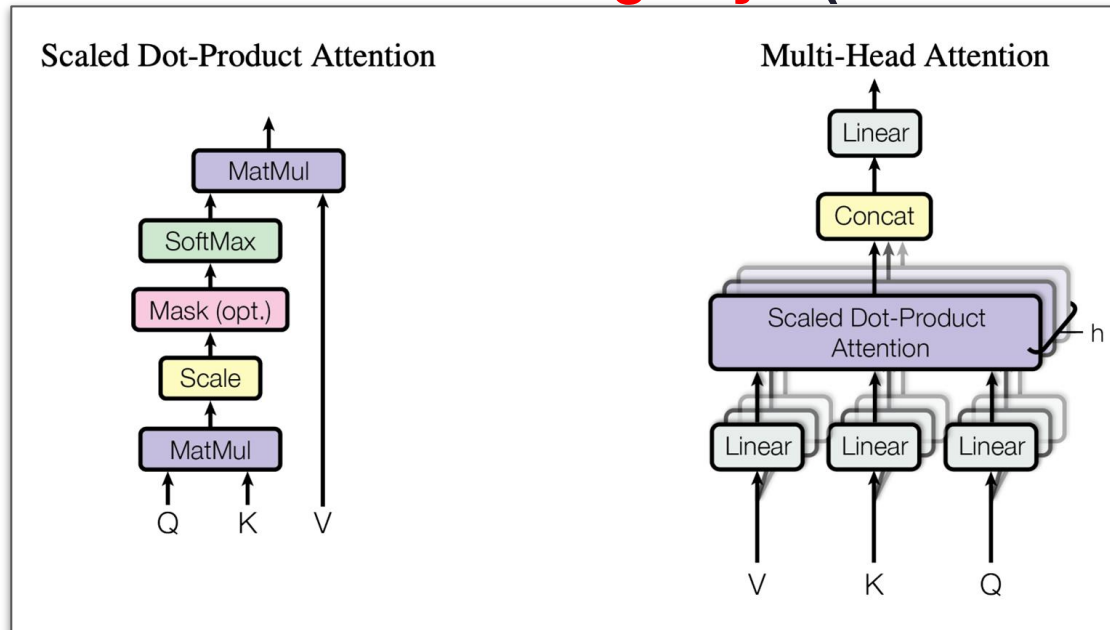
$$\tilde{\mathbf{e}}_m^{(h)} = \sum_{k=1}^M \alpha_{m,k}^{(h)} (\mathbf{W}_{\text{Value}}^{(h)} \mathbf{e}_k), \quad (6)$$

$$\tilde{\mathbf{e}}_m = \tilde{\mathbf{e}}_m^{(1)} \oplus \tilde{\mathbf{e}}_m^{(2)} \oplus \dots \oplus \tilde{\mathbf{e}}_m^{(H)}, \quad (7)$$

To preserve previously learned combinatorial features, including raw individual (i.e., first-order) features, we add standard residual connections in our network. Formally,

$$\mathbf{e}_m^{\text{Res}} = \text{ReLU}(\tilde{\mathbf{e}}_m + \mathbf{W}_{\text{Res}} \mathbf{e}_m), \quad (8)$$

제안한 방법 : **Interacting Layer**(이 논문의 핵심!)



출처 : [Song, cikm AutoInt\(2019\)](#)

1. 논문 리뷰

$$\alpha_{m,k}^{(h)} = \frac{\exp(\psi^{(h)}(\mathbf{e}_m, \mathbf{e}_k))}{\sum_{l=1}^M \exp(\psi^{(h)}(\mathbf{e}_m, \mathbf{e}_l))}, \quad (5)$$

$$\psi^{(h)}(\mathbf{e}_m, \mathbf{e}_k) = \langle \mathbf{W}_{\text{Query}}^{(h)} \mathbf{e}_m, \mathbf{W}_{\text{Key}}^{(h)} \mathbf{e}_k \rangle,$$

$$\tilde{\mathbf{e}}_m^{(h)} = \sum_{k=1}^M \alpha_{m,k}^{(h)} (\mathbf{W}_{\text{Value}}^{(h)} \mathbf{e}_k), \quad (6)$$

$$\tilde{\mathbf{e}}_m = \tilde{\mathbf{e}}_m^{(1)} \oplus \tilde{\mathbf{e}}_m^{(2)} \oplus \dots \oplus \tilde{\mathbf{e}}_m^{(H)}, \quad (7)$$

To preserve previously learned combinatorial features, including raw individual (i.e., first-order) features, we add standard residual connections in our network. Formally,

$$\mathbf{e}_m^{\text{Res}} = \text{ReLU}(\tilde{\mathbf{e}}_m + \mathbf{W}_{\text{Res}} \mathbf{e}_m), \quad (8)$$

제안한 방법 : **Interacting Layer**(이 논문의 핵심!)

- Input feature들간의 관계를 Multi-Head Self-Attention 구조로 모델링하여 의미있는 High order feature를 형성
- 서로 다른 Feature들간의 상관관계를 모델링
- Query, Key, Value 관계를 활용해서 Feature간의 관계를 모델링함
- Multi-Head 구조를 활용해서 Different subspace 상에서 구별된 Feature interaction을 분리하여 모델링
- Residual connections을 활용해 Raw individual feature 정보를 제공
 - 이전 정보를 보존하고 기억하기 위해

출처 : [Song, ckm AutoInt\(2019\)](#)

1. 논문 리뷰

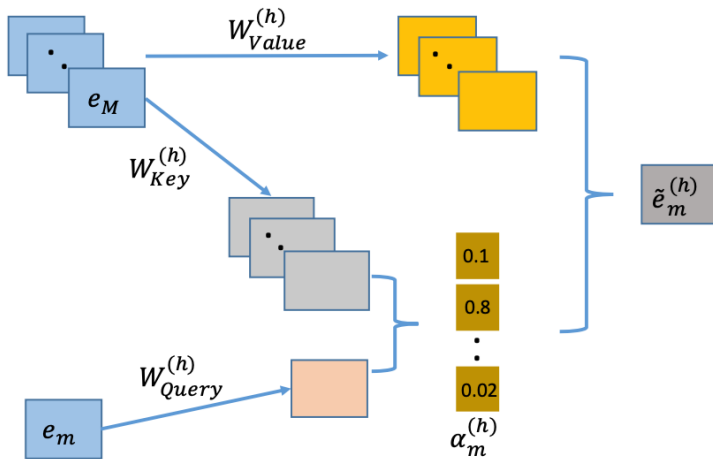


Figure 3: The architecture of interacting layer. Combinatorial features are conditioned on attention weights, i.e., $\alpha_m^{(h)}$.

출처 : [Song, cikm AutoInt\(2019\)](#)

제안한 방법 : **Interacting Layer**(이 논문의 핵심!)

- Input feature들간의 관계를 Multi-Head Self-Attention 구조로 모델링하여 의미있는 High order feature를 형성
- 서로 다른 Feature들간의 상관관계를 모델링
- Query, Key, Value 관계를 활용해서 Feature간의 관계를 모델링함
- Multi-Head 구조를 활용해서 Different subspace 상에서 구별된 Feature interaction을 분리하여 모델링
- Residual connections을 활용해 Raw individual feature 정보를 제공
 - 이전 정보를 보존하고 기억하기 위해

1. 논문 리뷰

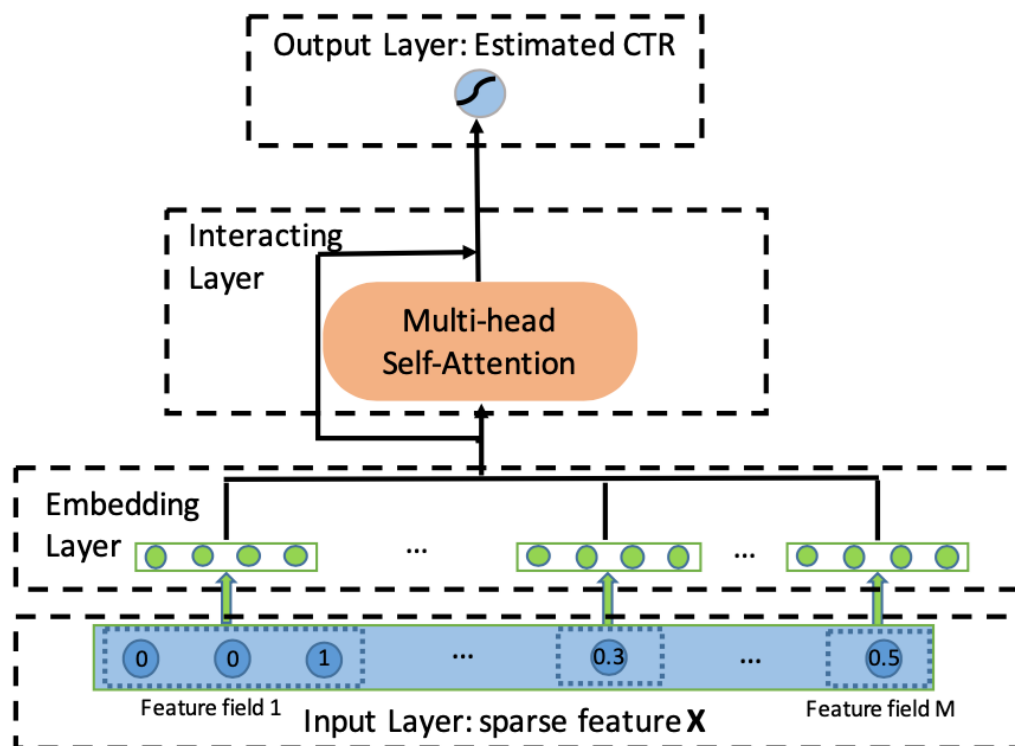


Figure 1: Overview of our proposed model AutoInt. The details of embedding layer and interacting layer are illustrated in Figure 2 and Figure 3 respectively.

출처 : [Song, cikm_AutoInt\(2019\)](#)

1. 논문 리뷰

Table 2: Effectiveness Comparison of Different Algorithms. We highlight that our proposed model almost outperforms all baselines across four data sets and both metrics. Further analysis is provided in Section 5.2.

Model Class	Model	Criteo		Avazu		KDD12		MovieLens-1M	
		AUC	Logloss	AUC	Logloss	AUC	Logloss	AUC	Logloss
First-order	LR	0.7820	0.4695	0.7560	0.3964	0.7361	0.1684	0.7716	0.4424
Second-order	FM [26]	0.7836	0.4700	0.7706	0.3856	0.7759	0.1573	0.8252	0.3998
	AFM[40]	0.7938	0.4584	0.7718	0.3854	0.7659	0.1591	0.8227	0.4048
High-order	DeepCrossing [32]	0.8009	0.4513	0.7643	0.3889	0.7715	0.1591	0.8448	0.3814
	NFM [13]	0.7957	0.4562	0.7708	0.3864	0.7515	0.1631	0.8357	0.3883
	CrossNet [38]	0.7907	0.4591	0.7667	0.3868	0.7773	0.1572	0.7968	0.4266
	CIN [19]	0.8009	0.4517	0.7758	0.3829	0.7799	0.1566	0.8286	0.4108
	HOFM [5]	0.8005	0.4508	0.7701	0.3854	0.7707	0.1586	0.8304	0.4013
	AutoInt (ours)	0.8061**	0.4455**	0.7752	0.3824	0.7883**	0.1546**	0.8456*	0.3797**

AutoInt outperforms the strongest baseline w.r.t. Criteo, KDD12 and MovieLens-1M data at the: ** 0.01 and * 0.05 level, unpaired t-test.

출처 : [Song, cikm_AutoInt\(2019\)](#)

1. 논문 리뷰

5.5 Integrating Implicit Interactions (RQ4)

Feed-forward neural networks are capable of modeling implicit feature interactions and have been widely integrated into existing CTR prediction methods [8, 11, 19]. To investigate whether integrating implicit feature interactions further improves the performance, we combine AutoInt with a two-layer feed-forward neural network by joint training. We name the joint model *AutoInt+* and compare it with the following algorithms:

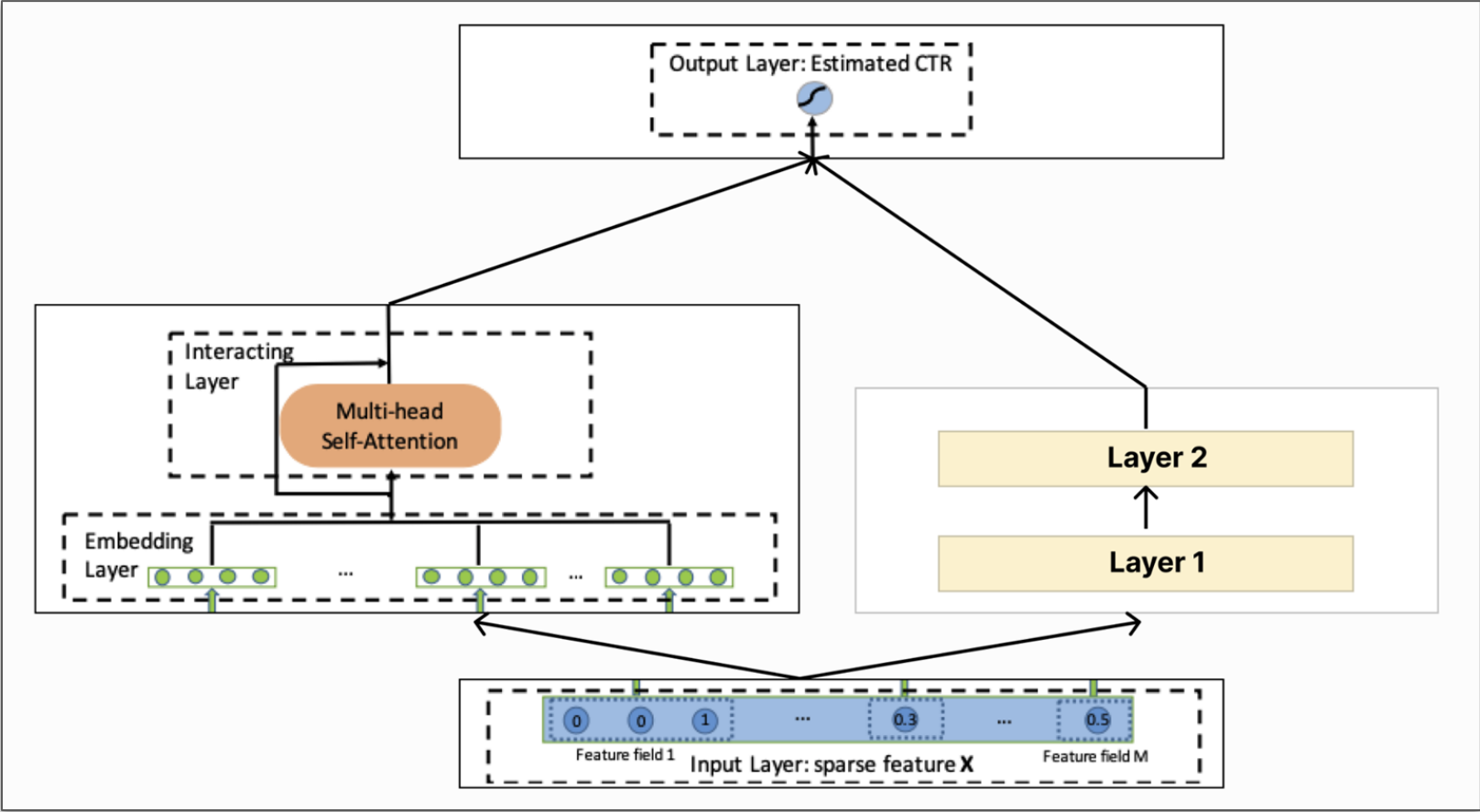
- Wide&Deep [8]. Wide&Deep integrates the outputs of logistic regression and feed-forward neural networks.
- DeepFM [11]. DeepFM combines traditional second-order factorization machines and feed-forward neural network, with a shared embedding layer.
- Deep&Cross [38]. Deep&Cross is the extension of CrossNet by integrating feed-forward neural networks.

출처 : [Song, ctkm, AutoInt\(2019\)](#)

제안한 방법 : **AutoInt+**

- 피드 포워드 뉴럴 네트워크 구조는 암묵적 피쳐 상호작용(implicit feature interaction)을 모델링 할 수 있음
- 기존 연구들은 암묵적 피쳐 상호작용(implicit feature interaction)을 통합하여 자주 활용하였음
- AutoInt에 2개의 레이어를 가진 피드포워드 뉴럴 네트워크를 결합해 조인트(Joint) 훈련 방법을 진행하고 이를 AutoInt+라 명명
- 이를 통해 성능 향상을 이끌어 낼 수 있었음

1. 논문 리뷰



출처 : [Song, cikm_AutoInt\(2019\)](#)

1. 논문 리뷰

Table 2 결과

AutoInt (ours)	0.8061**	0.4455**	0.7752	0.3824	0.7883**	0.1546**	0.8456*	0.3797**
----------------	----------	----------	--------	--------	----------	----------	---------	----------

Table 5: Results of Integrating Implicit Feature Interactions. We indicate the base model behind each method. The last two columns are average changes of AUC and Logloss compared to corresponding base models (“+”: increase, “-”: decrease).

Model	Criteo		Avazu		KDD12		MovieLens-1M		Avg. Changes	
	AUC	Logloss	AUC	Logloss	AUC	Logloss	AUC	Logloss	AUC	Logloss
Wide&Deep (LR)	0.8026	0.4494	0.7749	0.3824	0.7549	0.1619	0.8300	0.3976	+0.0292	-0.0213
DeepFM (FM)	0.8066	0.4449	0.7751	0.3829	0.7867	0.1549	0.8437	0.3846	+0.0142	-0.0113
Deep&Cross (CN)	0.8067	0.4447	0.7731	0.3836	0.7872	0.1549	0.8446	0.3809	+0.0200	-0.0164
xDeepFM (CIN)	0.8070	0.4447	0.7770	0.3823	0.7820	0.1560	0.8463	0.3808	+0.0068	-0.0096
AutoInt+ (ours)	0.8083**	0.4434**	0.7774*	0.3811**	0.7898**	0.1543**	0.8488**	0.3753**	+0.0023	-0.0020

AutoInt+ outperforms the strongest baseline w.r.t. each data at the: ** 0.01 and * 0.05 level, unpaired t-test.

출처 : Song, cikm_AutoInt(2019)

2. 코드 리뷰

proach not only outperforms existing state-of-the-art approaches for prediction but also offers good explainability. Code is available at: <https://github.com/DeepGraphLearning/RecommenderSystems>.

RecommenderSystems Public

Watch 33 Fork 273 Star 1k

master 1 branch 0 tags

Go to file Add file <> Code About

Songweiping casual rec e07370a on Apr 10, 2020 37 commits

File/Folder	Commit Message	Time Ago
featureRec	update bibtext	4 years ago
sequentialRec	initialize files	4 years ago
socialRec	initialize files	4 years ago
LICENSE	Create LICENSE	4 years ago
README.md	Update README.md	4 years ago
dataList.md	initialize files	4 years ago
readingList.md	casual rec	3 years ago

README.md

A library of Recommender Systems

This repository provides a summary of our research on Recommender Systems. It includes our code base on different recommendation topics, a comprehensive reading list and a set of benchmark data sets.

About
No description, website, or topics provided.

Releases
No releases published

Packages
No packages published

출처 : [AutoInt github](https://github.com/DeepGraphLearning/RecommenderSystems)

```
# ----- main part of AutoInt-----
self.y_deep = self.embeddings # None * M * d
for i in range(self.blocks):
    self.y_deep = multihead_attention(queries=self.y_deep,
                                       keys=self.y_deep,
                                       values=self.y_deep,
                                       num_units=self.block_shape[i],
                                       num_heads=self.heads,
                                       dropout_keep_prob=self.dropout_keep_prob[0],
                                       is_training=self.train_phase,
                                       has_residual=self.has_residual)

self.flat = tf.reshape(self.y_deep,
                       shape=[-1, self.output_size * self.field_size])

self.out = tf.add(tf.matmul(self.flat, self.weights["prediction"]),
                  self.weights["prediction_bias"], name='logits') # None * 1
```

2. 코드 리뷰

```
# ----- main part of AutoInt -----
self.y_deep = self.embeddings # None * M * d
for i in range(self.blocks):
    self.y_deep = multihead_attention(queries=self.y_deep,
                                      keys=self.y_deep,
                                      values=self.y_deep,
                                      num_units=self.block_shape[i],
                                      num_heads=self.heads,
                                      dropout_keep_prob=self.dropout_keep_prob[0],
                                      is_training=self.train_phase,
                                      has_residual=self.has_residual)

self.flat = tf.reshape(self.y_deep,
                       shape=[-1, self.output_size * self.field_size])

self.out = tf.add(tf.matmul(self.flat, self.weights["prediction"]),
                  self.weights["prediction_bias"], name='logits') # None * 1
```

출처 : [AutoInt github](#)

```
def multihead_attention(queries,
                        keys,
                        values,
                        num_units=None,
                        num_heads=1,
                        dropout_keep_prob=1,
                        is_training=True,
                        has_residual=True):

    if num_units is None:
        num_units = queries.get_shape().as_list[-1]

    # Linear projections
    Q = tf.layers.dense(queries, num_units, activation=tf.nn.relu)
    K = tf.layers.dense(keys, num_units, activation=tf.nn.relu)
    V = tf.layers.dense(values, num_units, activation=tf.nn.relu)
    if has_residual:
        V_res = tf.layers.dense(values, num_units, activation=tf.nn.relu)

    # Split and concat
    Q_ = tf.concat(tf.split(Q, num_heads, axis=2), axis=0)
    K_ = tf.concat(tf.split(K, num_heads, axis=2), axis=0)
    V_ = tf.concat(tf.split(V, num_heads, axis=2), axis=0)

    # Multiplication
    weights = tf.matmul(Q_, tf.transpose(K_, [0, 2, 1]))

    # Scale
    weights = weights / (K_.get_shape().as_list()[-1] ** 0.5)

    # Activation
    weights = tf.nn.softmax(weights)
```

2. 코드 리뷰

$$\alpha_{m,k}^{(h)} = \frac{\exp(\psi^{(h)}(\mathbf{e}_m, \mathbf{e}_k))}{\sum_{l=1}^M \exp(\psi^{(h)}(\mathbf{e}_m, \mathbf{e}_l))}, \quad 1 \quad (5)$$

$$\psi^{(h)}(\mathbf{e}_m, \mathbf{e}_k) = \langle \mathbf{W}_{\text{Query}}^{(h)} \mathbf{e}_m, \mathbf{W}_{\text{Key}}^{(h)} \mathbf{e}_k \rangle, \quad 2$$

출처 : AutoInt github

```
def multihead_attention(queries,
                        keys,
                        values,
                        num_units=None,
                        num_heads=1,
                        dropout_keep_prob=1,
                        is_training=True,
                        has_residual=True):

    if num_units is None:
        num_units = queries.get_shape().as_list[-1]

    # Linear projections
    Q = tf.layers.dense(queries, num_units, activation=tf.nn.relu)
    K = tf.layers.dense(keys, num_units, activation=tf.nn.relu)
    V = tf.layers.dense(values, num_units, activation=tf.nn.relu)
    if has_residual:
        V_res = tf.layers.dense(values, num_units, activation=tf.nn.relu)

    # Split and concat
    Q_ = tf.concat(tf.split(Q, num_heads, axis=2), axis=0)
    K_ = tf.concat(tf.split(K, num_heads, axis=2), axis=0)
    V_ = tf.concat(tf.split(V, num_heads, axis=2), axis=0)

    # Multiplication
    weights = tf.matmul(Q_, tf.transpose(K_, [0, 2, 1]))

    # Scale
    weights = weights / (K_.get_shape().as_list()[-1] ** 0.5)

    # Activation
    weights = tf.nn.softmax(weights)
```

2. 코드 리뷰

$$\tilde{\mathbf{e}}_{\mathbf{m}}^{(\mathbf{h})} = \sum_{k=1}^M \alpha_{\mathbf{m},k}^{(\mathbf{h})} (\mathbf{W}_{\text{Value}}^{(\mathbf{h})} \mathbf{e}_{\mathbf{k}}), \quad 1 \quad (6)$$

$$\tilde{\mathbf{e}}_{\mathbf{m}} = \tilde{\mathbf{e}}_{\mathbf{m}}^{(1)} \oplus \tilde{\mathbf{e}}_{\mathbf{m}}^{(2)} \oplus \dots \oplus \tilde{\mathbf{e}}_{\mathbf{m}}^{(\mathbf{H})}, \quad 2 \quad (7)$$

To preserve previously learned combinatorial features, including raw individual (i.e., first-order) features, we add standard residual connections in our network. Formally,

$$\mathbf{e}_{\mathbf{m}}^{\text{Res}} = \text{ReLU}(\tilde{\mathbf{e}}_{\mathbf{m}} + \mathbf{W}_{\text{Res}} \mathbf{e}_{\mathbf{m}}), \quad 3 \quad (8)$$

출처 : AutoInt github

4

```
# Dropouts
weights = tf.layers.dropout(weights, rate=1-dropout_keep_prob,
                             training=tf.convert_to_tensor(is_training))

# Weighted sum
outputs = tf.matmul(weights, V_) 1

# Restore shape
outputs = tf.concat(tf.split(outputs, num_heads, axis=0), axis=2) 2

# Residual connection
if has_residual:
    outputs += V_res 3

outputs = tf.nn.relu(outputs) 4
# Normalize
outputs = normalize(outputs)

return outputs
```

2. 코드 리뷰

AutoInt 모델을 리뷰해봤습니다.

**Multi-Head Self-Attention 구조를 이용해
강력한 성능을 보여줍니다.**

**프로젝트에서 사용할 모델이니
잘 배워두시면 좋을 것 같습니다.**

05

HAFP

모두의연구소

1. 논문 리뷰

1. 논문 리뷰

Hierarchical attention and feature projection for click-through rate prediction

Jinjin Zhang¹  · Chengliang Zhong² · Shouxiang Fan¹ · Xiaodong Mu¹ · Zhen Ni³

Accepted: 16 October 2021 / Published online: 2 November 2021

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2021

[출처 : Zhang, springer_HAFP\(2021\)](#)

본 연구는 논문만 살펴봅니다.

Feature interaction을 모델링하는 과정의 아이디어
를 얻을 수 있을 것 같아서 가지고 왔습니다.

1. 논문 리뷰

Hierarchical attention and feature projection for click-through rate prediction

Jinjin Zhang¹  · Chengliang Zhong² · Shouxiang Fan¹ · Xiaodong Mu¹ · Zhen Ni³

Accepted: 16 October 2021 / Published online: 2 November 2021
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2021

[출처 : Zhang, springer_HAFP\(2021\)](#)

조금 어려울 수도 있습니다.

괜찮습니다.

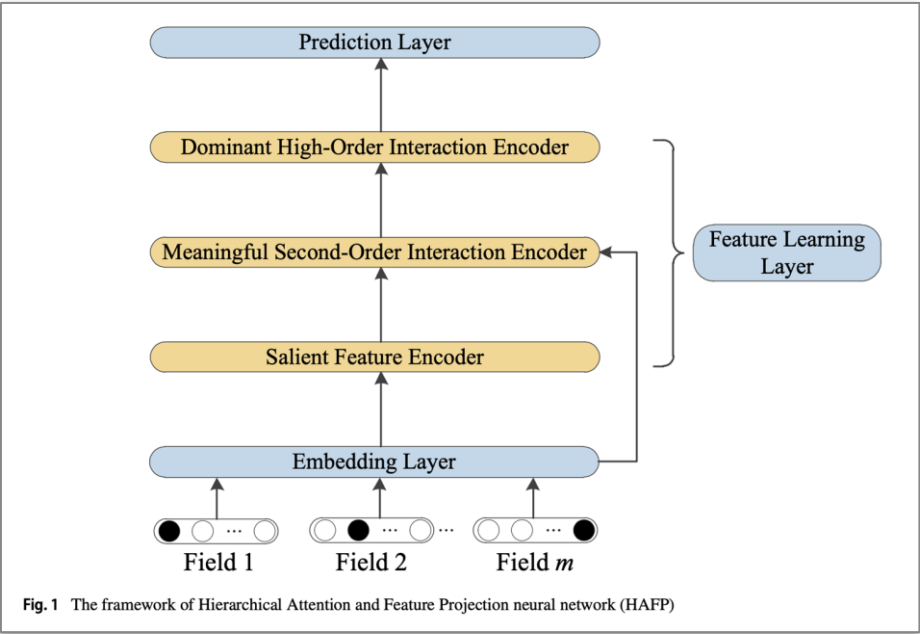
이 연구에서 ‘무엇’을 하고 싶은지만
중점적으로 보시고 이해하셔도 됩니다.

1. 논문 리뷰

본 논문의 주요 기여사항

1. 기존 연구들은 각 Feature의 중요성을 무시하거나 Feature interaction을 동등하게 다루었음
2. 또한, 내적이나 원소끼리 곱하는 연산은 Feature learning 학습에 있어 너무 간단함
3. 본 논문에서는 HAFP라는 모델을 제안
 - a. 계층적 구조(Hierarchical)의 어텐션 메커니즘을 활용
 - b. 2차 상호작용을 의미있게 변환하여 포괄적이고 세분화된 학습 가능
4. 본 논문의 핵심 구조는
 - a. Salient Feature Layer
 - b. Meaningful Second-order Interaction Encoder
 - c. Dominant High-Order Interaction Encoder

1. 논문 리뷰



출처 : [Zhang, springer_HAFP\(2021\)](#)

본 논문의 주요 기여사항

1. 기존 연구들은 각 feature의 중요성을 무시하거나 feature interaction을 동등하게 다루었음
2. 또한, inner product나 Hadamard product는 feature learning 학습에 있어 너무 간단함
3. 본 논문에서는 HAFP라는 모델을 제안
 - a. 계층적 구조(Hierarchical)의 어텐션 메커니즘을 활용
 - b. 2차 상호작용을 의미있게 변환하여 포괄적이고 세분화 된 학습 가능
4. 본 논문의 핵심 구조는
 - a. Salient Feature Layer
 - b. Meaningful Second-order Interaction Encoder
 - c. Dominant High-Order Interaction Encoder

1. 논문 리뷰

Salient Feature Encoder

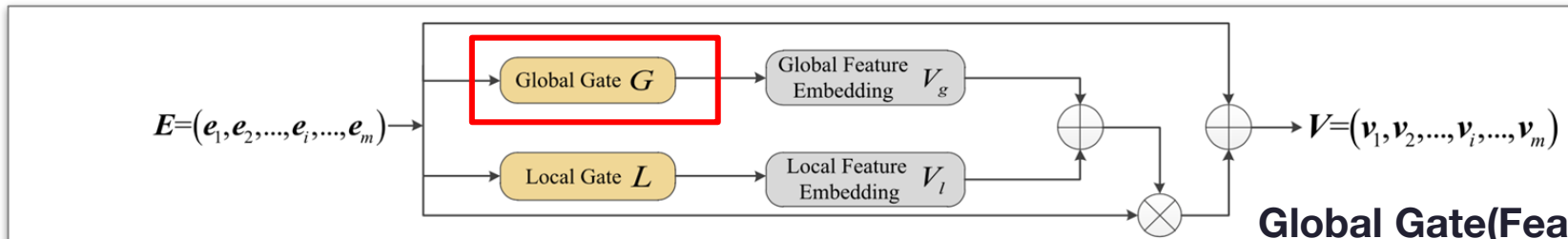


Fig. 2 The attentive global-local contexts module

Global Gate(Feature의 글로벌 어텐션 정보)

- Feature의 임베딩 벡터를 가져옴
- 각 임베딩에 Mean pooling을 적용해 글로벌(Global) 정보 획득(수식 1)
- 이후 뉴럴 네트워크 구조에 글로벌 정보를 넣어줌
 - 이때 생성된 값을 Global gate G값
 - 이게 Weight값
- 이를 임베딩과 곱하여 Weight 적용

$$a_i = \frac{1}{d} \sum_{j=1}^d e_i^j \quad (1)$$

$$G = [g_1, g_2, \dots, g_i, \dots, g_m] = \sigma_1(W_{g1}\sigma_2(W_{g2}A)) \quad (2)$$

$$\begin{aligned} V_g &= [v_{g1}, v_{g2}, \dots, v_{gi}, \dots, v_{gm}] \\ &= [g_1 \cdot e_1, g_2 \cdot e_2, \dots, g_i \cdot e_i, \dots, g_m \cdot e_m] \end{aligned} \quad (3)$$

출처 : [Zhang, springer_HAFP\(2021\)](#)

1. 논문 리뷰

Salient Feature Encoder

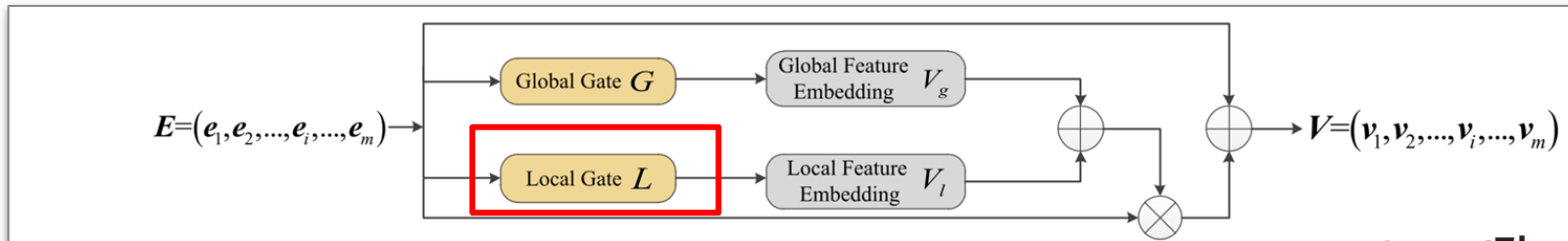


Fig. 2 The attentive global-local contexts module

Local Gate (각 Feature가 가지고 있는 특징 정보)

- Feature의 임베딩 벡터를 가져옴
- 이후 뉴럴 네트워크 구조에 벡터를 넣어줌
 - 이때 생성된 값을 Local gate L값
 - 이게 Weight값
- 이를 임베딩과 곱하여 Weight 적용

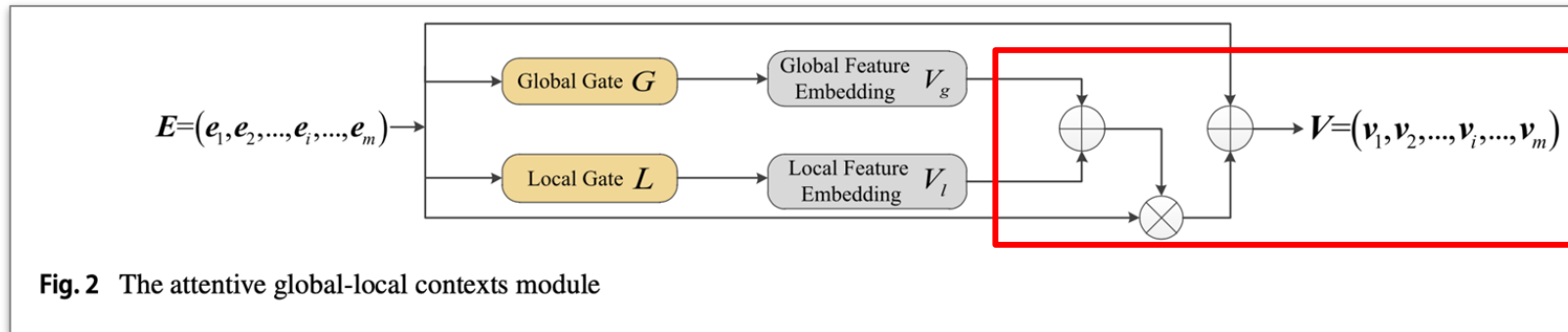
$$L = [l_1, l_2, \dots, l_i, \dots, l_m] = \sigma_1(W_{l1}\sigma_2(W_{l2}E)) \quad (4)$$

$$V_l = [v_{l1}, v_{l2}, \dots, v_{li}, \dots, v_{lm}] = [l_1 \cdot e_1, l_2 \cdot e_2, \dots, l_i \cdot e_i, \dots, l_m \cdot e_m] \text{ the } \text{pro}$$

출처 : [Zhang, springer_HAFP\(2021\)](#)

1. 논문 리뷰

Salient Feature Encoder



$$V = [v_1, v_2, \dots, v_i, \dots, v_m] = (E \otimes \sigma(V_l \oplus V_g)) \oplus E \quad (6)$$

출처 : [Zhang, springer_HAFP\(2021\)](#)

- 최종적으로 Local gate, Global gate 값을 전부 이용해 Salient feature vector를 구함
- 즉 Feature의 중요도를 Global 관점과 Local 관점에서 구하는 것

1. 논문 리뷰

Meaningful second-order interaction encoder

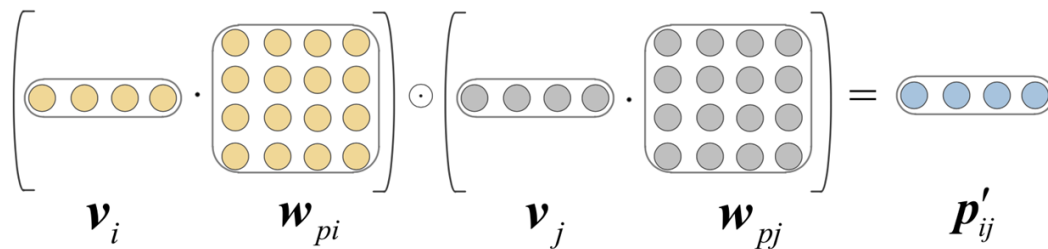


Fig. 3 The projective bilinear function for feature interaction

$$p'_{ij} = (v_i \cdot W_{pi}) \odot (v_j \cdot W_{pj}) \quad (7)$$

$$\alpha_{ij}^p = \frac{\exp(h_p^T \text{ReLU}(W_{ij}^p p'_{ij} + b_p))}{\sum_{i,j} h_p^T \text{ReLU}(W_{ij}^p p'_{ij} + b_p)} \quad (8)$$

$$p_{ij} = \alpha_{ij}^p p'_{ij} \quad (9)$$

출처 : Zhang, springer HAFP(2021)

Second order - p

- 피쳐의 중요도를 반영한 Salient encoder 결과를 가져옴
 - 내적(Inner product)을 통해 i와 j 사이의 Feature들의
- Second order feature interaction 수행(식 7)
- 어텐션 값을 구함(식 8)
 - 연관성 있는 Feature를 강조하기 위함

1. 논문 리뷰

Meaningful second-order interaction encoder

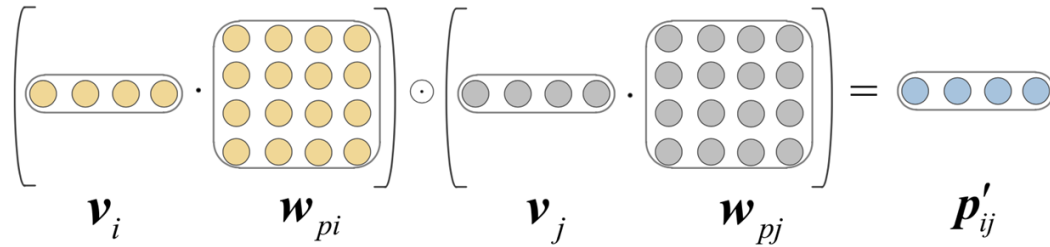


Fig. 3 The projective bilinear function for feature interaction

$$q'_{ij} = (e_i \cdot W_{qi}) \odot (e_j \cdot W_{qj}) \quad (10)$$

$$\alpha_{ij}^q = \frac{\exp(h_q^T \text{ReLU}(W_{ij}^q q'_{ij} + b_q))}{\sum_{i,j} h_q^T \text{ReLU}(W_{ij}^q q'_{ij} + b_q)} \quad (11)$$

$$q_{ij} = \alpha_{ij}^q q'_{ij} \quad (12)$$

출처 : Zhang, springer_HAFP(2021)

Second order - q

- p를 구했음에도, Residual connection 효과를 보기 위한 목적으로 q를 구함
- 내적을 통해 i와 j 사이의 Feature들의 Second order feature interaction 수행(식 10)
- 어텐션 값을 구함(식 11)
 - 연관성 있는 Feature를 강조하기 위함

1. 논문 리뷰

Meaningful second-order interaction encoder

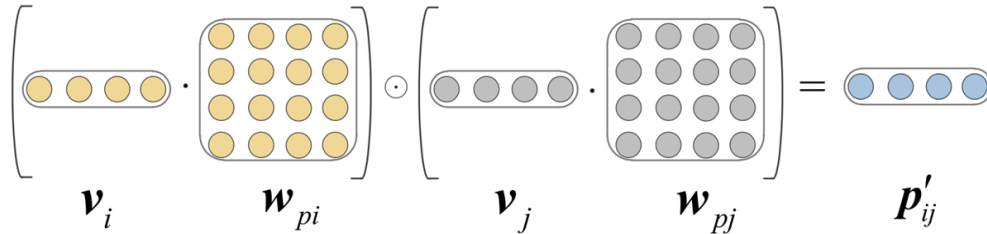


Fig. 3 The projective bilinear function for feature interaction

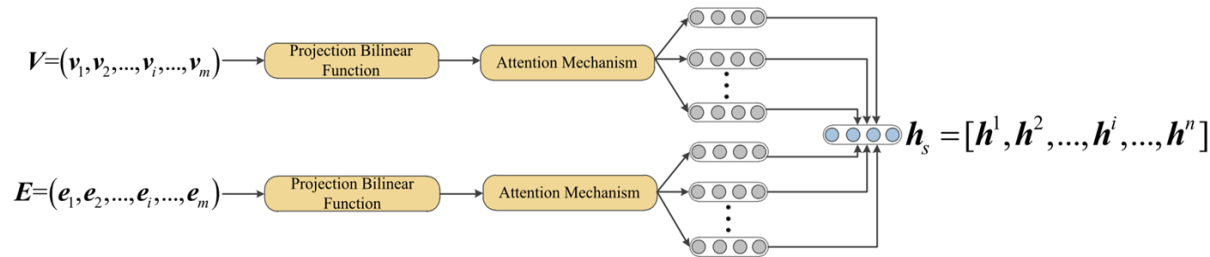


Fig. 4 The meaningful second-order interaction encoder

$$\begin{aligned}
 h_s &= [h^1, h^2, \dots, h^i, \dots, h^n] \\
 &= FC(\text{Concat}(p_1, p_2, \dots, p_i, \dots, p_n, q_1, q_2, \dots, q_i, \dots, q_n)W_{fc})
 \end{aligned}
 \tag{13}$$

인코더의 최종 Output

- 이러한 과정으로 2차 상호작용을 좀 더 풍부하게 만들어 줌
- 구한 값들을 전부 Concat해준 뒤 Fully connected layer를 통과하여 포괄적인 정보를 학습

1. 논문 리뷰

Dominant high-order interaction encoder

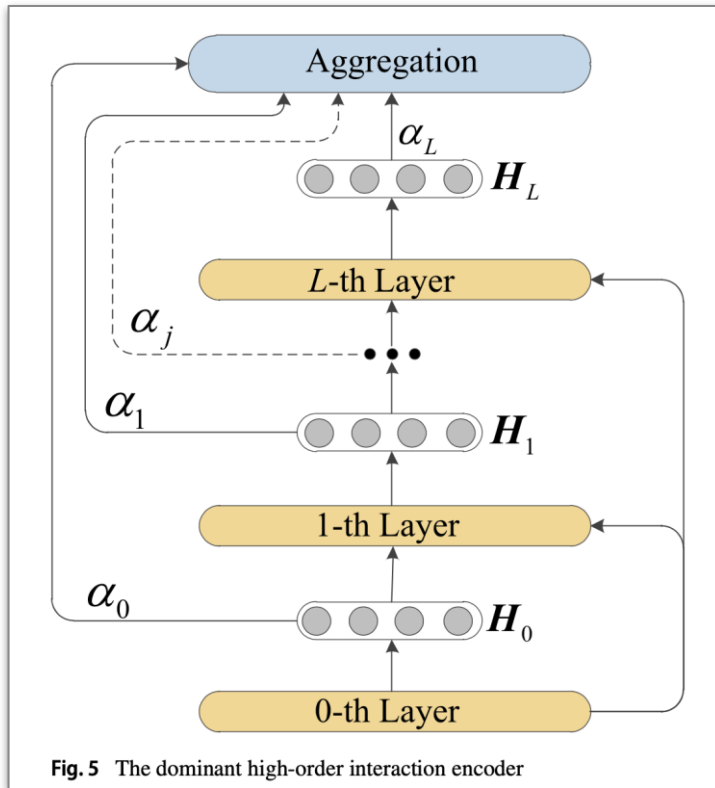


Fig. 5 The dominant high-order interaction encoder

출처 : Zhang, springer_HAFP(2021)

$$H_1 = \sigma(W_0 H_0 + b_0) \quad (14)$$

$$H_L = \sigma(W_{L-1} H_{L-1} + b_{L-1}) \quad (15)$$

$$\alpha_k = \frac{\exp(h_k \text{ReLU}(W_k H_k + b_k))}{\sum_{k=0}^L \exp(h_k \text{ReLU}(W_k H_k + b_k))} \quad (16)$$

$$h = [\alpha_0 H_0, \alpha_1 H_1, \dots, \alpha_L H_L] \quad (17)$$

High order interaction을 학습하기 위한 목적의 인코더

- 계층적 구조를 활용해 Layer 사이의 관계를 반영함
- Layer마다 나오는 Feature 관계는 서로 다를 것이기 때문
- 어텐션 메커니즘을 활용해 관련성 있는 정보만 담도록 함

1. 논문 리뷰

정리하자면, 이 논문의 전반적인 흐름은
‘의미있는, 연관성 있는 feature를 잘 추출해서 활용해보겠다’
입니다.

본 논문에서 무엇을 하고 싶은지를 이해하고,
이렇게도 할 수 있구나 정도의
인사이트만 얻어가셔도 됩니다.

핵심 정리

이번 챕터에서는 다음과 같은 내용을 학습하였습니다.

- 추천 시스템의 평가 방법과 데이터 셋 특징
- NCF 논문 리뷰와 코드 리뷰
- DeepFM 논문 리뷰와 코드 리뷰
- AutoInt 논문 리뷰와 코드 리뷰
- HAFP 논문 리뷰

다음 시간 소개

다음 시간은 **프로젝트**입니다.

AutoInt 모델을 기본으로 사용하여 프로젝트를 실행해 봅시다.

감사합니다.

딥러닝 기반 추천 시
추천시스템, 모델 튜닝: 논문과 코드

이수진